Claude Code for Everyone Else

The complete beginner's guide to the most powerful coding agent in the world

Marco Kotrotsos

February 2026

Contents

PART 1

UNDERSTANDING

1

What Is Claude Code?

You have probably used ChatGPT, or Claude, or one of the other AI chatbots. You type a question, it types back an answer. Maybe you asked it to write an email, or explain a concept, or help you brainstorm. That is a conversation. It lives inside a browser window, and when you close the tab, nothing has changed on your computer.

Claude Code is something different.

Claude Code is an AI agent that works directly on your computer. It reads files, creates files, edits files, runs programs, and builds things. When you tell Claude Code to make you a website, you do not get a block of text to copy and paste somewhere. You get actual files on your computer that you can open in a browser and see a real, working website.

That is the difference between chatting with AI and having an AI agent. A chatbot gives you answers. An agent does work.

What Can It Actually Build?

The short answer: quite a lot. Here are things people are building with Claude Code right now, with zero programming experience.

Personal websites. A portfolio page with your name, bio, photo, and links to your work. Looks professional. Takes about ten minutes. You describe what you want, approve the files Claude Code creates, and open the result in your browser. Done.

Small business sites. A landing page for a freelance practice, a local bakery, a consulting firm. Multiple pages, contact forms, embedded maps. One person built a complete restaurant website with a menu, reservation system, and photo gallery in under an hour.

Internal tools. A calculator that estimates project costs based on your rates. A form that collects data and saves it to a spreadsheet. A simple dashboard that shows numbers you care about. These are the kinds of tools that companies pay thousands of dollars to have custom-built, and you can create a working version in an afternoon.

Automation scripts. A program that renames 500 files according to a naming convention. A tool that pulls data from one place and formats it for another. Things that would take you hours to do by hand, done in seconds. If you have ever

stared at a repetitive task and thought "a computer should be doing this," Claude Code can make that happen.

Simple apps. A habit tracker. A recipe organizer. A quiz for your students. A workout log. A budget tracker. Nothing that would replace Instagram, but genuinely useful tools that solve a specific problem you have.

The common thread is that these are all real, functioning things that live on your computer (or on the internet, if you choose to publish them). Not suggestions. Not instructions you need to follow. Finished products, created by an AI agent working on your behalf.

The Chatbot vs. Agent Distinction

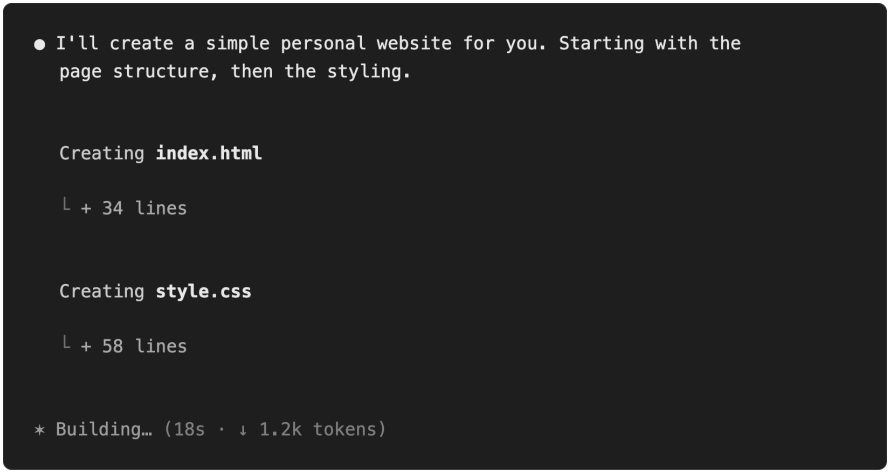
This distinction matters, and being precise about it helps.

When you use a chatbot, the loop looks like this: you ask a question, the AI responds with text, you read the text, you decide what to do with it. If you asked the chatbot to write you a website, it would give you a block of code. Then you would need to figure out where to save it, what to name the file, how to open it, and how to make it actually work. The chatbot cannot touch your computer. It can only talk.

When you use Claude Code, the loop looks different: you describe what you want, Claude Code reads your existing files (if any), writes new code, creates the files on your computer, and can even run the result to check that it works. You see what it is doing at each step, and you can say yes or no to each action.

But the agent does the work, not you.

Think of it this way. A chatbot is like calling a friend who is a mechanic and asking what is wrong with your car. They can give you advice, but you still have to pick up the wrench. Claude Code is like that same mechanic showing up at your garage, diagnosing the problem, getting the parts, and doing the repair while you watch and approve each step.



Claude Code running in terminal, creating files for a website project

You Do Not Need to Understand the Code

This is the part that trips people up. If Claude Code writes code, do you need to understand code?

No.

You need to understand what you want. You describe it in plain English (or any language you are comfortable with). Claude Code translates that into code. You do not need to read the code, understand the code, or even look at the code. You look at the result. Does the website look the way you wanted? Does the tool calculate the right numbers? Does the form collect the information you need?

If the answer is yes, you are done. If the answer is no, you tell Claude Code what to change, and it changes it. The code is a means to an end, and Claude Code handles that part.

This is not a simplified or watered-down experience. It is the same tool professional developers use. You just interact with it differently. A developer might say "refactor the authentication middleware to use JWT tokens." You might say "add a contact form that sends me an email when someone fills it out." Both are valid instructions. Claude Code handles both.

What you will develop over time is a sense of how to describe what you want clearly. That is a skill, but it is not a programming skill. It is more like learning to give good instructions to a contractor. "Make the kitchen nicer" is vague. "Replace the countertops with butcher block and paint the cabinets white" gives the contractor something to work with. Same idea with Claude Code.

The code is still there, in files on your computer, fully accessible if you ever want to look at it. Some people start recognizing patterns over time, noticing that `<h1>` means a big heading, or that `color: blue` means what it sounds like. That happens naturally with exposure, but it is never a requirement. You can

use Claude Code for months or years without ever reading a line of code.

Why This Matters for Non-Developers

There are roughly 30 million software developers in the world. There are roughly 8 billion people. That means the ability to turn ideas into working software has been locked behind a skill that 99.6% of people do not have.

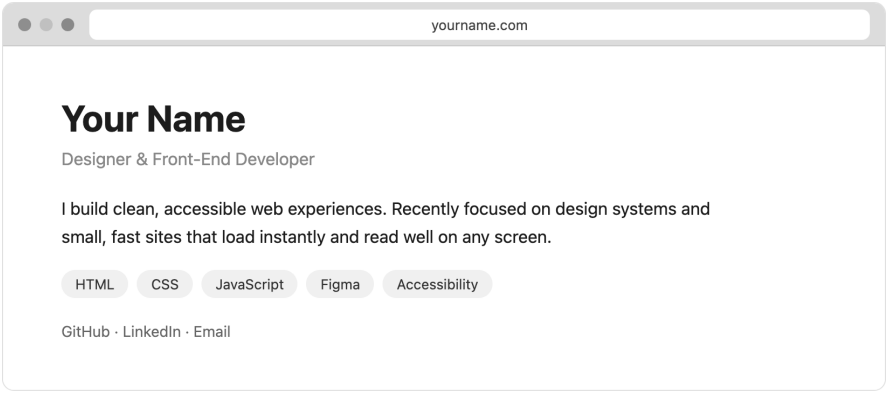
Claude Code changes that ratio. Not by teaching everyone to code (though you will pick up bits and pieces along the way), but by making the coding part something an AI handles for you. Your contribution is the idea, the direction, the judgment about what is good enough. That is the part that actually requires a human.

You might be a teacher who wants to build a classroom tool. A small business owner who needs a simple app for customers. A researcher who wants to automate a repetitive data task. A creative person who has a vision for something interactive. None of these require you to become a programmer. They require you to have a clear idea of what you want and the willingness to spend some time working with Claude Code to get there.

Before Claude Code, the options for non-developers were limited. You could hire a developer (expensive, slow, requires you to communicate your vision through someone else). You

could use a no-code tool like Squarespace or Wix (limited to what the tool's designers anticipated). You could learn to code yourself (months or years of effort before building anything useful). Or you could let the idea sit in your head, unrealized.

Claude Code creates a fifth option: describe what you want, let the AI build it, judge the result, and refine from there. The cost is a monthly subscription. The time investment is the length of a conversation. The ceiling is higher than you would expect.



A personal portfolio website running in a browser, built entirely through a Claude Code conversation. Clean design, professional-looking, no code visible

The quality of what Claude Code produces is genuine. Websites it builds look professional. Tools it creates work correctly. The gap between "built by an AI agent on my computer" and "built by a junior web developer" is small and shrinking. For most personal and small business use cases, it is more than good enough.

This book walks you through all of it, from opening the terminal for the first time to building and running your own projects. The learning curve is real but short. Most people are building

something useful within their first hour.

That starts with understanding one tool you have probably never opened: the terminal. The next section introduces it, and you will find it is a lot less intimidating than it looks.

2

What Is a Terminal?

The terminal is an app on your computer. That is all it is. It has been there the whole time, sitting in your Applications folder, waiting for someone to open it.

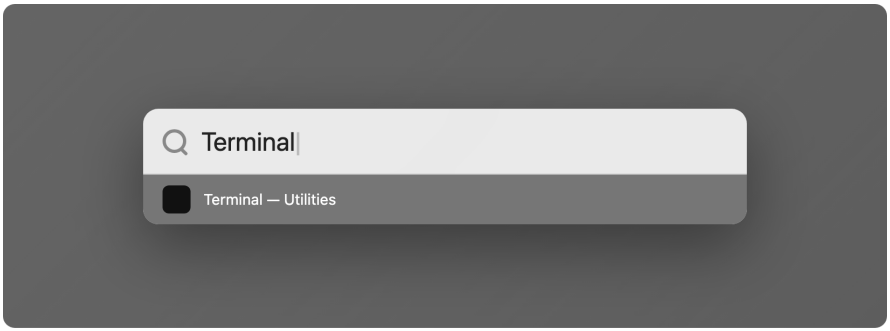
When you open it, you see a mostly empty window with a blinking cursor. No buttons, no menus to speak of, no colorful interface. Just a text prompt, waiting for you to type something. It looks like it belongs in a movie from the 1980s, and in a way it does. Terminals have been around for decades. The reason they still exist is that they are extremely good at what they do.

Finding It on Your Mac

There are a few ways to open Terminal, but the fastest one is Spotlight.

1. Press Cmd+Space on your keyboard. A search bar appears in the middle of your screen.

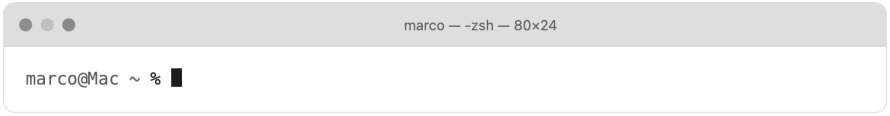
2. Type "Terminal" and press Enter. The Terminal app opens.



Spotlight search bar with "Terminal" typed in

That is it. Two steps.

If you prefer clicking, you can also find it by opening Finder, navigating to Applications, then Utilities, and double-clicking Terminal. But the Spotlight method is faster and you will use it more often.



Terminal app freshly opened, showing the default prompt

What You Are Looking At

When Terminal opens, you see a line of text that looks something like this:

```
marco@MacBook-Pro ~ %
```

This might look slightly different on your machine. The username will be yours, the computer name will be whatever

your Mac is called. But the structure is the same: some identifying information, followed by a % or $ sign, followed by a blinking cursor.

The % is just the prompt character. It is the terminal's way of saying "I am ready. Type something." On some Macs you will see $ instead. The difference does not matter at all for our purposes. Both mean the same thing: your turn.

The ~ symbol you see represents your home directory. Think of it as "you are here" marker, like the star on a mall map. When you first open the terminal, you start in your home folder, which is the same place that contains your Desktop, Documents, and Downloads folders.

The Mall Map Analogy

A terminal is like navigating a building with spoken directions instead of a visual map.

In Finder (the file browser you normally use), you navigate by clicking folder icons. You can see everything laid out visually. You click Documents, you see what is inside. You click a subfolder, you go deeper.

In the terminal, you do the same thing with short typed commands. Instead of clicking a folder to open it, you type a command and press Enter. The result is text, not icons. But you are doing the exact same thing: moving between folders and looking at what is inside them.

Finder is the mall map. The terminal is the information desk where you ask a person for directions. Same mall. Different way of getting around.

Five Commands You Need

You only need a handful of commands for everything in this book. Here they are.

Where Am I? (pwd)

```
pwd
```

This stands for "print working directory." It tells you which folder you are currently in. When you type `pwd` and press Enter, the terminal shows you the full path:

```
/Users/marco
```

That path reads like a home address: start at the root of the computer (`/`), go into the Users folder, then into the marco folder. That is where you are.

> **Tip:**
> If you ever feel lost in the terminal, type `pwd`.
> It always tells you exactly where you are.

What Is Here? (ls)

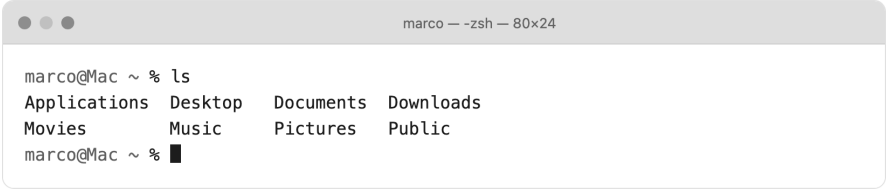
```
ls
```

This stands for "list." It shows you what is inside your current folder. When you type `ls` and press Enter, you see a list of files and folders:

```
Desktop    Documents    Downloads    Music    Pictures
```

These are the same folders you see when you open a Finder window. They are just displayed as text instead of icons.

If the list is empty, the folder is empty. Not an error, just nothing in there yet.



Terminal showing output of ls command in home directory

Go Somewhere (cd)

```
cd Documents
```

This stands for "change directory." It moves you into another folder. After typing `cd Documents`, you are now inside the Documents folder. If you type `pwd` again, you will see:

```
/Users/marco/Documents
```

To go back up one level (back to where you were), type:

```
cd ..
```

The two dots mean "parent folder," which is the folder that contains this one. Think of it like pressing the back button.

To jump straight back to your home folder from anywhere, type:

```
cd ~
```

That ~ is shorthand for your home directory. It does not matter how deep you have navigated, `cd ~` always takes you home.

Make a Folder (mkdir)

```
mkdir my-project
```

This stands for "make directory." It creates a new folder with whatever name you give it. After running this command, if you type `ls`, you will see `my-project` listed among your other files and folders.

If you open Finder and navigate to the same location, you will see the folder there too. The terminal and Finder are looking at the same files. Anything you create in one shows up in the other.

You can then move into it:

```
cd my-project
```

This is how you will set up every project in this book: make a folder, move into it, then start Claude Code.

Clear the Screen (clear)

```
clear
```

This one is cosmetic. It clears all the text off the screen so you have a fresh view. It does not delete anything or undo anything. It just scrolls the previous output out of sight. Useful when things start looking cluttered.

Trying It Out

Go ahead and open Terminal right now. Then run these commands, one at a time, pressing Enter after each:

```
pwd
```

You should see your home directory path.

```
ls
```

You should see familiar folder names like Desktop, Documents, Downloads.

```
mkdir terminal-test
```

This creates a new folder called "terminal-test."

```
cd terminal-test
```

Now you are inside that folder.

```
pwd
```

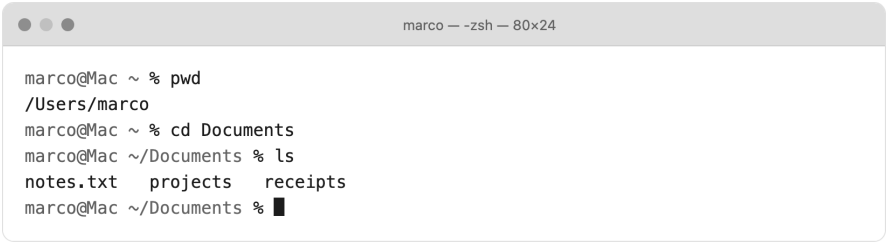
Confirms you are in `/Users/yourname/terminal-test`.

```
ls
```

Shows nothing, because the folder is brand new and empty.

```
cd ~
```

Takes you back to your home directory.



Terminal session showing the above commands executed in sequence

If all of that worked, you just used the terminal. Every command you will need for the rest of this book is a variation of these five. The specific folder names will change, the specific commands will be different, but the pattern of type a command, press Enter, read the result stays the same throughout.

A Bonus Command: open

There is one more command that bridges the gap between the terminal and the visual world you are used to:

```
open .
```

The dot means "this folder." So `open .` tells your Mac to open the current folder in a Finder window. It is handy when you want to see your project files visually, or when you want to double-click a file to open it in the normal way.

You can also open specific files. If Claude Code creates a webpage called `index.html`, you can type `open index.html` and it opens in your browser. We will use this later when building your first project.

The Tab Trick

One small thing that will save you a lot of typing: the Tab key auto-completes. If you are trying to type `cd Documents` but you have only typed `cd Doc`, press Tab. The terminal fills in the rest for you.

If there are multiple options that start with the same letters, press Tab twice and it shows you all the possibilities. This is not essential, but once you start using it, you will find it hard to stop.

It Really Is Just a Text Box

The reason the terminal looks intimidating is that it offers no visual clues about what you can do. There are no buttons labeled "create folder" or "move to this directory." You have to know the command, or look it up. That is the entire learning curve.

But the things the terminal can do are the same things you already do in Finder every day: make folders, move between folders, look at files, run programs. The terminal is not a separate world. It is a different interface to the same computer you already know.

You might be wondering why Claude Code uses this interface instead of something with buttons and menus. The next section answers that question, and the answer might change how you think about the terminal entirely.

3

Why the Terminal?

A reasonable question. It is 2026. We have touchscreens and voice assistants and apps with beautiful interfaces. Why does the most powerful coding agent in the world run in a text window from the 1970s?

Because text is faster, and faster matters when an AI is doing work for you.

The Speed of Text

Think about how you would ask a friend to rearrange your living room. You could draw a floor plan, mark where each piece of furniture should go, add measurements and arrows. Or you could just say "swap the couch and the bookshelf, and move the coffee table closer to the window."

The second version communicates the same thing in a fraction of the time. That is what the terminal offers: a direct line of communication between you and Claude Code, with nothing in between. No menus to navigate. No dropdowns to find. No buttons to click in the right order. You type what you want, Claude Code does it.

A graphical app would need a button for every possible action. Make a file. Rename a file. Move a file. Edit a file. Delete a file. Choose a template. Pick a color. Set a font. That is a lot of buttons. And every new feature means more buttons, more menus, more interface to learn.

The terminal replaces all of that with one thing: a place to type. Since Claude Code understands plain English, the interface can stay simple while the capabilities grow without limit.

Consider a practical example. You want to build a website and then change the background color to dark blue. In a graphical tool, you would click through a color picker, find the right shade, apply it, check the preview. In the terminal with Claude Code, you type "change the background color to dark blue." Same result, fewer steps, and Claude Code applies the change across every file that needs it. You do not need to know which files those are.

Claude Code Can Do More Through Text

There is a technical reason too. When Claude Code works through the terminal, it has direct access to your computer's file system, can run any command, can install software, can start servers, can interact with version control, and can chain multiple actions together without waiting for you to click through each step.

A graphical app would need to expose each of those capabilities through a carefully designed interface. That takes months of design and development work, and the result is always a compromise, some actions are easy, some are buried three menus deep, and some are not available at all.

The terminal has no such constraints. If a computer can do it, the terminal can do it. And if the terminal can do it, Claude Code can do it. That openness is the whole point.

Five Commands, Not Fifty

The worry people have about terminals is usually about memorization. The idea that you need to learn hundreds of commands and remember arcane syntax.

You do not. For everything in this book, you need five commands:

1. `pwd` to check where you are 2. `ls` to see what files are around you 3. `cd` to move to a different folder 4. `mkdir` to create a new folder 5. `claude` to start Claude Code

That is the complete list. You just learned the first four in the previous section. The fifth one you will learn in a few pages. Once Claude Code is running, you go back to typing plain English.

If you can type a URL into a browser's address bar, you can use the terminal. The mental model is the same: type a specific thing, press Enter, get a result. The only difference is that the browser shows you a webpage and the terminal shows you text.

Both respond to what you type.

Professional Developers Use the Same Tool

One thing that might surprise you: the terminal you are about to use is not a beginner's version of a real tool. It is the real tool. Professional software developers at Google, Apple, Meta, and every startup you have heard of use the exact same terminal on the exact same Mac.

The difference is that they have memorized hundreds of commands over years of practice, and you are going to use five of them plus an AI agent that handles the rest. You arrive at the same destination through a different path. But the tool is the same.

It Stops Being Scary Fast

Most people who have never used a terminal expect it to feel alien and difficult. In practice, the feeling goes away after about fifteen minutes.

Once you have typed `cd` a few times and seen it work, it stops being mysterious. It is just typing a short word and pressing Enter. The same gesture you make a thousand times a day in text messages, emails, and search bars. The terminal is just one more place to type.

There is also no penalty for mistakes. If you type a command wrong, the terminal tells you it does not understand. Nothing breaks. Nothing gets deleted. You just try again. The worst case scenario in 99% of situations is an error message, and even those are usually clear enough to figure out.

By the time you finish the next section, you will have installed Claude Code, logged in, and had your first conversation with it. You will have typed real commands into a real terminal. And you will understand, from direct experience, that this is not as complicated as it looked from the outside.

The hard part is opening the terminal for the first time. You already did that. The next section covers the short checklist of what you need before installing Claude Code. Spoiler: you probably have everything already.

PART 2

GETTING SET UP

4

What You Need Before Starting

This section is a quick checklist. Nothing to install yet, nothing to configure. Just make sure you have these things, and you are ready for the next section where we actually set everything up.

A Mac

Claude Code runs on macOS 13.0 or later. That is Ventura, Sonoma, or Sequoia. If you bought your Mac in the last three or four years, you almost certainly have a compatible version.

To check, click the Apple menu in the top-left corner of your screen, then click "About This Mac." The version number is right

there.

> **Note:**
> Claude Code also works on Windows and Linux. This book focuses on Mac because that is where most beginners start, but the concepts are nearly identical on other systems. If you are on Windows, you will use PowerShell or WSL instead of Terminal. If you are on Linux, you already know what a terminal is.

An Internet Connection

Claude Code talks to Claude's servers every time you use it. There is no offline mode. If your internet drops mid-conversation, it will pick up where it left off once you reconnect, but you do need a stable connection to work.

A standard home Wi-Fi connection is more than enough. You do not need anything fast or special.

A Claude Account

This is the one thing that costs money. Claude Code requires a paid Claude subscription. The free plan does not include access.

Your options:

- **Pro ($20/month):** The starting point for most people. Gives you solid access to Claude Code with Sonnet and Opus models.

Good enough for everything in this book and beyond.

- **Max ($100 or $200/month):** Higher usage limits and full access to the most powerful model (Opus 4.6). You do not need this to start, but heavy users sometimes upgrade later.

If you already have a Claude account for chatting on claude.ai, check which plan you are on. If you are on the free tier, you will need to upgrade to at least Pro before installing Claude Code.

Sign up or upgrade at claude.ai/pricing.

> **Tip:**
> Pro and Max plans share usage limits between the Claude website and Claude Code. If you use Claude heavily on the web during the day, you may hit limits faster when switching to Claude Code in the evening. Something to keep in mind, not a dealbreaker.

About 15 Minutes

The installation itself takes about two minutes. The rest is logging in, verifying things work, and running your first conversation. Fifteen minutes is generous, but it accounts for anything unexpected.

Find a time when you are not rushed. You want a stretch where you can follow along step by step without interruptions. The process is simple, but doing it the first time requires a bit of focus. After that, starting Claude Code takes about five seconds.

Pouring coffee first is optional but recommended.

No Coding Experience Required

This might be the most important item on the list. You do not need to know how to code. You do not need to have ever written a line of code. You do not need to understand what HTML, Python, or JavaScript mean.

Claude Code writes the code. You describe what you want in plain English. The entire point of this book is that the barrier between "idea" and "working software" just got a lot smaller.

If you have been told you need to learn to code before using developer tools, that advice is outdated. You need to learn to communicate clearly with an AI that codes. That is a different skill, and you are about to learn it.

Some people worry that they will break something or mess up their computer. That is understandable, but Claude Code asks permission before it does anything significant. It will not delete files, install random software, or change settings without your explicit approval. You are in control at every step.

The Checklist

Before moving to the next section, confirm:

- [ ] Mac running macOS 13.0 or later
- [ ] Working internet connection

- [ ] Claude Pro account (or higher) at claude.ai
- [ ] 15 uninterrupted minutes
- [ ] Willingness to type a few commands into a text box

That is it. No special software to pre-install, no files to download in advance, no accounts to create on other platforms. Claude Code's installer handles everything.

That is everything. The next section is the one where you actually install Claude Code, step by step, with screenshots for every stage.

5

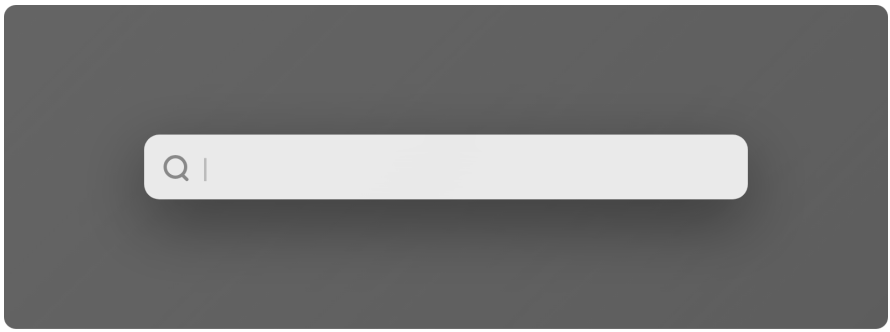
Installing Claude Code, Step by Step

This is the section where you actually do something. Every step has a screenshot placeholder so you know exactly what you should see on your screen. If something looks different from what is described, do not panic. Scroll down to the troubleshooting section at the end.

Take it slow. Read each step fully before doing it.

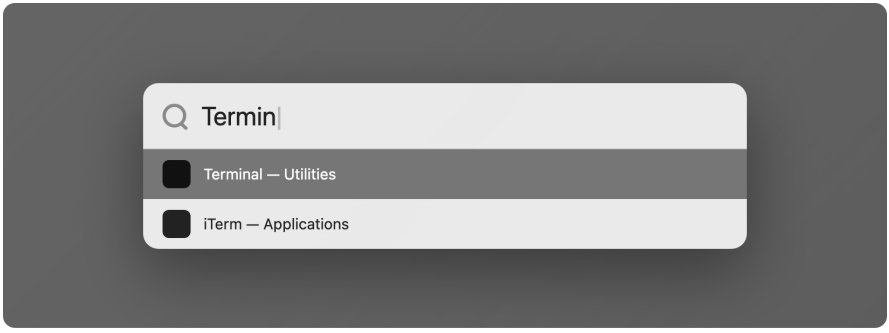
Step 1: Open Terminal

Press `Cmd` + `Space` on your keyboard. This opens Spotlight Search, the built-in search bar on your Mac.



Spotlight Search bar appearing in the center of the screen

Type the word `Terminal` and press `Enter`.

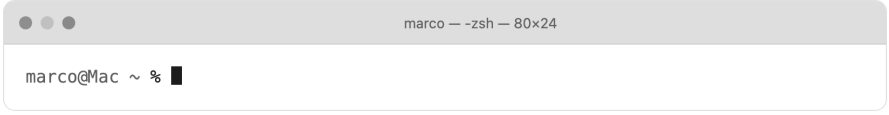


Spotlight showing Terminal as the top result

A window opens. It is mostly empty, with a line of text that looks something like this:

```
marco@MacBook-Pro ~ %
```

Your line will look slightly different. It shows your username, your computer's name, and a `%` sign. That `%` is the prompt. It means "ready for you to type something."



A fresh Terminal window with the default prompt

That is it. You just opened Terminal. If you went through Section 2, this should feel familiar. If you skipped ahead, no worries. All you need to know is: you type commands after the `%` sign and press `Enter` to run them.

Step 2: Run the Install Command

This is the step that installs Claude Code on your Mac. You are going to type a command, and it might look intimidating if you have never done this before. That is normal. Just type it exactly as shown, or even better, copy and paste it.

To copy and paste: highlight the command below with your mouse, press `Cmd + C` to copy, click in Terminal after the `%` prompt, and press `Cmd + V` to paste.

Here is the command:

```
curl -fsSL https://claude.ai/install.sh | bash
```

The install command typed into Terminal, before pressing Enter

Before you press `Enter`, here is what this command means, word by word. You do not need to memorize any of this, but understanding what you are running is always a good idea:

- **curl** is a tool built into every Mac. It downloads things from the internet. Think of it as a behind-the-scenes web browser that fetches files instead of web pages.
- **-fsSL** are options that tell curl to be quiet about what it is doing, follow any redirects, and fail cleanly if something goes wrong. You do not need to memorize these.

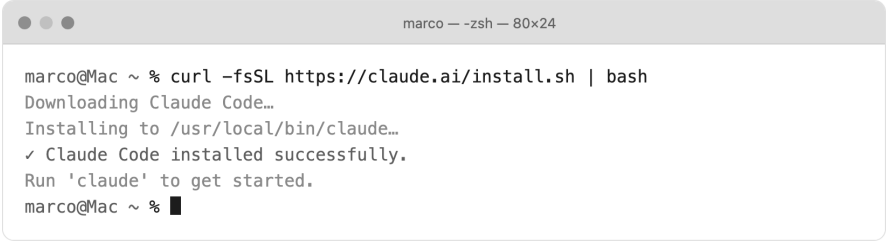
- **https://claude.ai/install.sh** is the web address of the installation script. This is the file curl downloads.
- | is called a "pipe." It takes whatever curl downloaded and feeds it directly into the next command.
- **bash** runs the downloaded script. Bash is the program that understands and executes shell scripts.

In plain English: "Download the Claude Code installer from Anthropic's website and run it."

Press `Enter`.

You will see text scroll by in the Terminal. This is the installer doing its work. It is downloading files, putting them in the right places, and configuring things. The whole process usually takes less than thirty seconds on a normal internet connection.

Do not touch anything while it runs. Just wait until you see a message confirming the installation succeeded and a new % prompt appears, meaning the command is done and Terminal is ready for your next input.



Terminal showing the install command output, ending with a success message

> **Note:**
> If you see an error at this step, the most common cause is a network issue. Check your internet connection and try again. If you are on a corporate network with a firewall or VPN, you may need to ask your IT team to allow connections to claude.ai. You can also try disconnecting from the VPN temporarily.

Step 3: What Just Happened

The installer did three things, and knowing what they are helps even if you never need to think about them again:

1. **Downloaded the Claude Code program** from Anthropic's servers. This is the actual software that runs when you type `claude`. 2. **Placed it in a folder** on your Mac at `~/.local/bin/claude`. The ~ means your home folder (something like `/Users/yourname`). The `.local` folder is hidden, which is why you will not see it in Finder. The dot at the beginning of the folder name is what makes it hidden. This is standard practice for developer tools, not something sneaky. 3. **Updated your shell configuration** so your Mac knows where to find the `claude` command. Without this step, you would have to type the full path to the program every time. Instead, you can just type `claude` from any folder and your Mac will know what you mean.

You do not need to memorize any of this. You do not need to find these folders or files. Just know that the installation is

clean, self-contained, and does not scatter files all over your system.

One thing that is genuinely nice about this installation method: Claude Code auto-updates in the background. You will not need to re-run the installer or manually update when new versions come out. When Anthropic releases a new version, your copy upgrades itself quietly. One less thing to think about.

Step 4: Verify It Works

Before going further, confirm the installation actually worked. Type this command and press `Enter`:

```
claude doctor
```

Terminal showing the claude doctor command and its output

`claude doctor` is a health check. It looks at your installation and tells you if everything is set up correctly. You should see output that mentions the version number and confirms things are working.

If you see something like `command not found: claude`, it means your terminal does not know where Claude Code is yet. The fix is simple: close Terminal completely (press `Cmd + Q`, not just the red X button), then reopen it and try `claude doctor`

again. Closing and reopening loads the updated configuration that the installer created.

Successful claude doctor output showing version and status

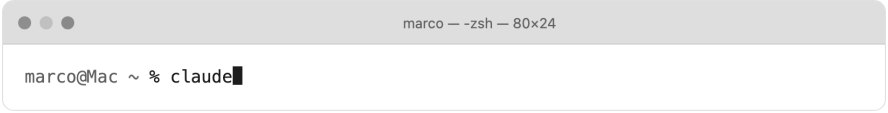
> **Tip:**
> If `claude doctor` still does not work after restarting Terminal, try running this command: `source ~/.zshrc` (or `source ~/.bashrc` if you use bash). This manually reloads your configuration without needing to close Terminal.

Step 5: Start Claude Code for the First Time

This is the moment. One word, five letters. Type:

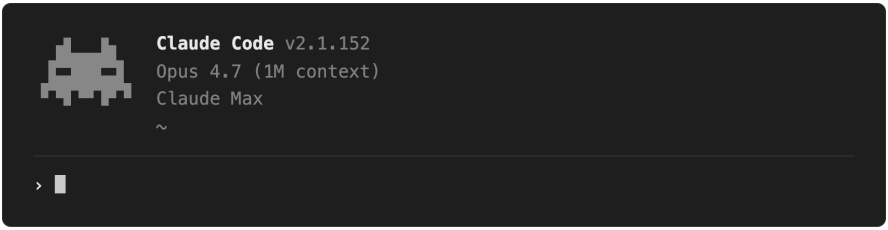
```
claude
```

And press `Enter`.



Terminal after typing "claude" and pressing Enter

Claude Code starts up. You will see a welcome screen with some information about the current version, the AI model it is using, and a text prompt at the bottom where you can type.



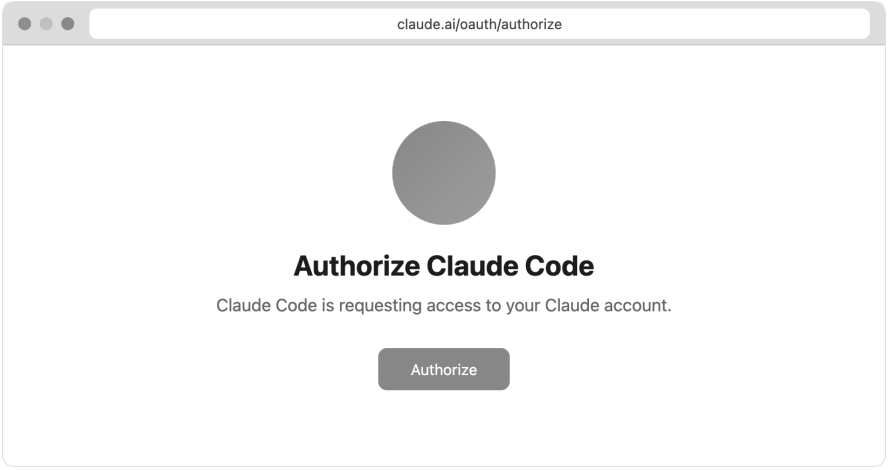
Claude Code welcome screen with the input prompt visible

The interface is entirely text-based. No buttons, no dropdown menus, no windows to drag around. Just a prompt waiting for your input, similar to a messaging app but without the chat bubbles. If you are used to graphical apps like Word or Photoshop, this might feel stripped down. That is by design. The simplicity is the point. Everything happens through conversation.

You might also notice a grayed-out suggestion next to the cursor. Claude Code tries to guess what you might want to ask based on the folder you are in. You can press `Tab` to accept the suggestion, or just start typing your own message. Ignore it for now.

Step 6: Log In

The first time you start Claude Code, it needs to connect to your Claude account. A browser window should open automatically, taking you to an authentication page on claude.ai.



Browser opening with the Claude authentication page

1. Sign in with your Claude account. Use the same email and password you use at claude.ai. If you use Google or Apple sign-in, those work too.

2. Approve the connection. The page will ask you to authorize Claude Code to use your account. Click "Allow" or "Authorize."

3. Return to Terminal. After authorizing, you can close the browser tab. Switch back to Terminal. Claude Code should now show that you are logged in and ready.

Terminal showing Claude Code after successful authentication

Your credentials are stored locally on your Mac. You will not need to log in again unless you explicitly log out or switch accounts.

What If the Browser Does Not Open?

This happens sometimes, especially if your default browser is not set up or you are using a non-standard configuration. If the browser does not open automatically:

1. Look at the Terminal output. Claude Code will show a message with instructions. 2. Press c to copy the authentication URL to your clipboard. 3. Open your browser manually and paste the URL into the address bar. 4. Complete the sign-in process as described above.

Terminal showing the "press c to copy URL" message

Troubleshooting

If everything went smoothly, skip this section. Come back to it if you run into issues later.

"command not found: claude"

You typed `claude` but Terminal does not recognize it. This usually means the shell configuration was not loaded yet.

Fix: Close Terminal completely with `Cmd + Q` and reopen it. Then try `claude` again. If that does not work, re-run the installer:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Then restart Terminal again.

"Permission denied"

The installer could not write to the necessary folders on your Mac.

Fix: Check that your Mac user account has normal administrator privileges. You can verify this by going to System Settings > Users & Groups and checking that your account says "Admin" underneath it. If you are on a work-issued Mac with restricted permissions, you may need your IT department to assist with the installation.

The Install Seems to Work but Claude Code Will Not Start

This is rare, but possible if your Mac has a non-standard shell configuration or an older operating system. Try running `claude doctor` first to see what it reports. The output usually identifies the specific issue and suggests a fix.

If `claude doctor` also does not work, the nuclear option is to re-run the entire installation from Step 2. The installer is designed to be safe to run multiple times, and it will overwrite the previous installation cleanly.

Authentication Keeps Failing

If the browser login does not seem to connect back to Terminal:

1. Make sure you are signing in with the correct account (the one with a Pro or Max subscription). 2. Try running `/logout` inside Claude Code, then restart with `claude` and log in again. 3. If all else fails, press `c` when prompted to manually copy the authentication URL.

Still Stuck?

You can always restart from scratch. The installation does not leave anything behind that would cause problems. Re-run the install command, restart Terminal, and try again. For persistent issues, check the troubleshooting page at code.claude.com/docs/en/troubleshooting.

That is it. Claude Code is installed, verified, and logged in. You have gone from "I have never used a terminal" to "I have a professional AI coding agent running on my Mac." The next section walks you through your first real conversation with Claude Code, what the interface looks like, how the permission system works, and how to get comfortable before building anything.

6

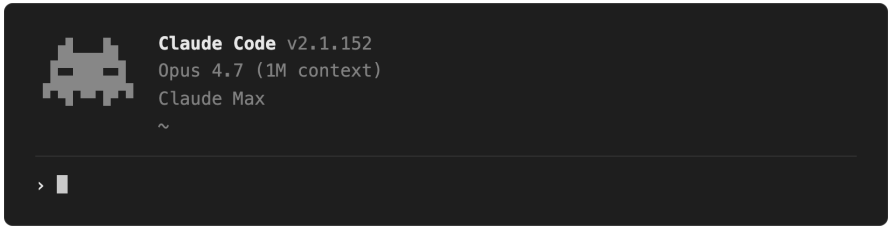
Your First Conversation

You just typed `claude` and pressed Enter. Claude Code is running. A prompt sits there, blinking, waiting for you to say something.

This is the part where most guides would tell you to build something immediately. We are not going to do that. First, you need to get comfortable with how Claude Code actually works. Think of this as a five-minute orientation before you start building.

What You Are Looking At

The Claude Code interface is a text prompt inside your Terminal. That is it. No sidebar, no file browser, no toolbars. You type a message in plain English, press Enter, and Claude responds.



Claude Code running with an empty prompt, ready for input

At the bottom of the screen, you might see a status line showing the model name (like "Opus 4.6" or "Sonnet 4.5") and other minor details. You can ignore this for now.

You might notice the grayed-out suggestion mentioned in the previous section. Same idea: press `Tab` to accept it, or just start typing your own message.

Try Saying Hello

Type this and press Enter:

```
what can Claude Code do?
```

```
› what can Claude Code do?

● I can help you build and change software by working directly in your
  project. A few things I do well:

  • Create and edit files (websites, scripts, configs)

  • Run commands and start your project

  • Explain code and fix errors you paste in

  • Remember project context in a CLAUDE.md file

Tell me what you want to make and I'll get started.
```

Claude Code responding to "what can Claude Code do?" with a summary of capabilities

Claude will respond with a summary of its capabilities. The response appears line by line, streaming in as Claude thinks through it. You will see that it can create files, edit code, run commands, search through projects, and more. The response might be long, several paragraphs covering different categories of things it can do. That is fine. Scroll through it if needed using your mouse or the arrow keys.

What you are looking at is not a pre-written help page. Claude generated that response specifically for you, in this moment, based on your question. If you ask a different question, you get a different response. It is a conversation, not a lookup table.

This first exchange does something important: it proves everything works. You typed a question in plain English, and an AI agent understood it and answered. No special syntax. No commands to memorize. Just a sentence.

There is a particular feeling that comes with this first interaction. It is the moment where the abstract idea of "an AI that works on your computer" becomes concrete. You typed words. Something happened. The gap between asking and getting a response is usually just a few seconds. If you have used ChatGPT or Claude on the web, the speed will feel familiar. The difference is that this version can do things beyond just responding with text.

Every interaction from here follows the same pattern. You describe what you want or ask a question, and Claude responds. Sometimes with text, sometimes by creating files, sometimes by running commands on your behalf. But it always starts the same way: you type, you press Enter.

The Permission System

This is the one thing you need to understand before doing anything else.

Claude Code asks for permission before it changes things on your computer. Not every action requires approval, but many do. Here is the logic:

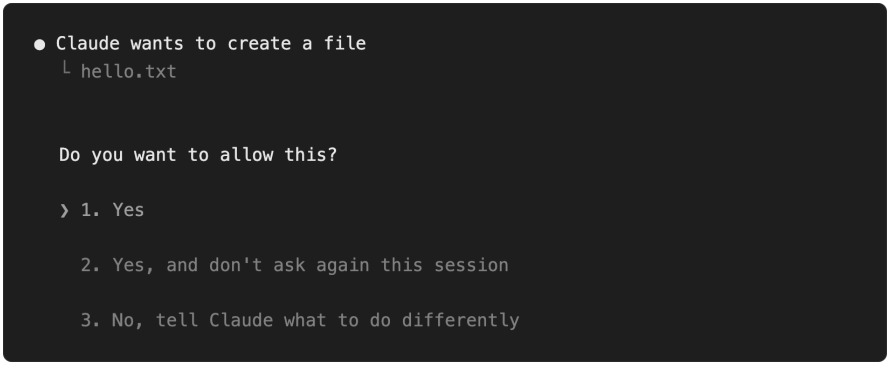
Things Claude does without asking:

- Reading files on your computer
- Searching through folders
- Looking at code

Things Claude asks about first:

- Editing or creating files
- Running commands in your terminal
- Installing packages or dependencies

When Claude wants to do something that needs your approval, you will see a prompt. It describes what Claude wants to do and gives you options: approve it, deny it, or let Claude run it automatically going forward.



A permission prompt showing Claude asking to create a file, with approve/deny options

This system exists to keep you in control. Claude Code is powerful, but it will not rewrite your files or run random commands without your say-so. For beginners, this is exactly what you want. You see what is happening before it happens.

> **Tip:**
> If Claude asks to do something and you are not sure what it means, just say no. Nothing bad happens. Claude will explain what it was trying to do, and you can decide whether to let it try again.

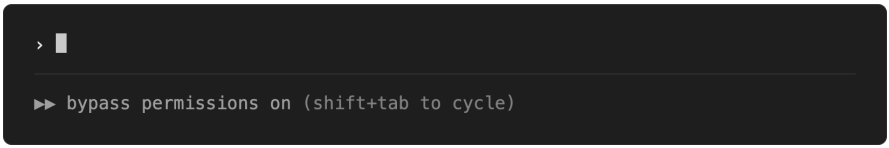
Permission Modes

Claude Code has three modes that change how strictly it asks for permission. You cycle through them by pressing `Shift + Tab`. Each press switches to the next mode, and the current mode shows at the bottom of the screen.

Normal mode is the default. Claude asks before editing files or running commands. You see every action before it happens and choose whether to allow it. This is where you should stay while learning.

Auto-accept mode lets Claude edit files without asking for each one. It still asks before running shell commands (because commands can do more than just edit files), but file creation and editing happens automatically. This speeds things up once you trust the process and want a more fluid workflow. You will probably switch to this mode after a few sessions.

Plan mode is read-only. Claude can read your files, search through folders, and think through a plan, but it absolutely cannot change anything. Nothing gets created, edited, or deleted. This is useful when you want Claude to analyze a situation or propose a strategy before you commit to changes.



The status indicator showing the current permission mode

For now, stay in Normal mode. It is the safest way to learn, and you will see exactly what Claude is doing at each step. Once you

are comfortable (probably by Section 9 or 10 in this book), you can experiment with the other modes.

Getting Comfortable

Try a few more things to build your confidence. If you are in a folder with files in it, ask:

```
what files are in this folder?
```

Claude will list the contents of whatever directory you are currently in. Notice that it does this without asking permission, since reading files is always allowed automatically.

Claude listing the contents of a folder

Try asking something more open-ended:

```
explain what this folder contains
```

If there are files, Claude will read through them and give you a summary of what they do. If the folder is empty (which it

probably is if you started Claude Code from your home directory), Claude will tell you that too. Either way, you will see how Claude processes a request and formulates a response.

You can also try follow-up questions. After Claude explains the folder contents, you could ask "which of these files is the most important?" or "is there anything unusual here?" Claude remembers everything you have discussed in the current session, so each message builds on the previous ones.

The point of these small interactions is to build a feel for the back-and-forth rhythm. You ask, Claude responds. You ask for more detail, Claude provides it. You change your mind, Claude adjusts. It is a conversation, not a form you fill out. There is no wrong thing to type.

Useful Things to Know Right Now

A few practical details that will save you confusion. You do not need to memorize all of these. Just read through them so you know they exist, and come back when you need a reference.

Multi-line input. If you want to type a longer message that spans multiple lines, press `\` then `Enter` (or `Option + Enter` on Mac) to add a new line without sending the message. This is useful when you want to give Claude detailed instructions with several points, rather than cramming everything into one line.

Canceling. If Claude is working on something and you want to stop it mid-response, press `Ctrl + C`. This interrupts whatever it is doing and gives you back control immediately. Think of it as

a stop button. You can always ask Claude to try again or take a different approach.

Clearing the screen. If the Terminal is getting cluttered with text and you want a clean view, press `Ctrl + L`. This clears the visible screen but keeps your entire conversation intact. It is purely visual.

Exiting. When you are done for the day, press `Ctrl + D` or type `/exit` to close Claude Code. You will be back at your normal Terminal prompt. From there you can close Terminal entirely, or start Claude Code again with `claude`.

Resuming. If you close Claude Code and want to continue where you left off later, type `claude -c` (the `-c` stands for "continue"). It picks up your most recent conversation in the current folder. This is helpful when you are working on something over multiple sessions and do not want to re-explain context.

Getting help. If you are ever unsure what commands or options are available, type `/help` inside Claude Code. It shows a list of built-in commands. You can also just ask Claude directly: "what commands can I use?" works fine.

A Note on Patience

Claude Code sometimes takes a few seconds to respond, especially for complex requests. You will see a small indicator when it is thinking. Do not press Enter again or type another message while it is working. Just wait. If it seems stuck for more

than a minute, press `Ctrl + C` to cancel and try rephrasing your question.

Also, Claude is not always perfect. It might misunderstand what you meant, or suggest something slightly different from what you had in mind. That is normal. Just clarify. Say "no, I meant..." or "can you change that to..." and Claude will adjust. The best results come from iterating, not from getting the perfect prompt on the first try.

What Happens Next

You now have Claude Code installed and you have had your first conversation. You know how the interface works, how permissions function, and how to ask questions.

The next section introduces one more concept before you start building: what a "project" is in Claude Code terms. It is simpler than you think, just a folder on your computer. Once that clicks, you will be ready to build your first real thing.

PART 3

BUILDING THINGS

7

What Is a Project?

You have Claude Code installed. You have had your first conversation with it. You know how to open Terminal and type commands. The next step is understanding where your work lives.

A project is a folder on your computer. That is it.

Not a special file format. Not something you create inside an application. Not a database or a cloud service. A folder. The same kind of folder you already have for photos, documents, or downloads.

Creating a Project

When you want to build something new, you create a new folder for it. Open Terminal and type:

```
mkdir my-website
```

You already learned `mkdir` in Section 2. It creates a new folder with whatever name you give it. If you just opened Terminal, you are probably in your home directory, so the new folder lives at `/Users/yourname/my-website`. If you want your projects in a specific location, navigate there first with `cd` before running `mkdir`.

Some people like to keep all their projects in one place. You could create a parent folder for everything:

```
mkdir projects
cd projects
mkdir my-website
```

Now `my-website` lives inside `projects`. This is optional, just a way to stay organized if you plan to build more than one thing.

To go inside your new project folder:

```
cd my-website
```

You are now inside your project.

If you want to verify where you are, type:

```
pwd
```

You will see something like `/Users/yourname/my-website` or `/Users/yourname/projects/my-website`. That confirms you are in the right place.

Why Folders Matter to Claude Code

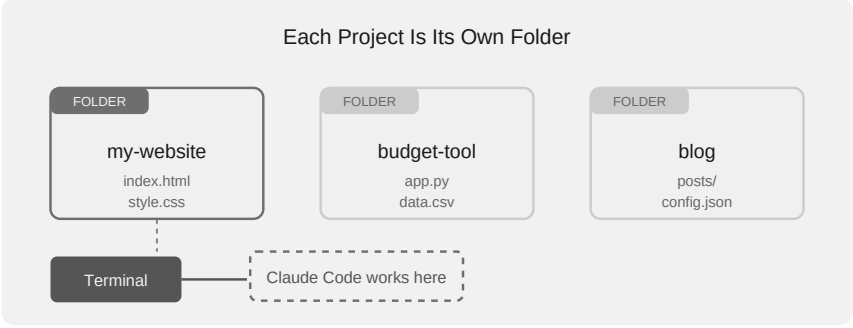
Claude Code works inside whatever folder you start it from. When you type `claude` in your Terminal, it looks at the folder it is in and treats that as its workspace. Every file it creates, every change it makes, happens inside that folder.

This is a big deal, and understanding it early will save you confusion later. Claude Code uses the folder as its entire context for the conversation. It reads the files inside the folder to understand what your project is, what has already been built, and what needs to happen next. If the folder is empty, Claude Code knows you are starting from scratch. If the folder has files from a previous session, Claude Code reads those files and understands the current state of your project without you explaining anything.

Think of it like walking into an office. If you start Claude Code from your Documents folder, it treats your entire Documents folder as the project. Every file in Documents becomes fair game. That is usually too broad. You do not want Claude Code looking at your tax returns when you asked it to build a website.

This is why you create a dedicated folder for each thing you build. One folder for your personal website. A different folder for that spreadsheet tool you want to make. Another for the blog you are working on. Each folder is its own self-contained project. Clean boundaries mean Claude Code stays focused on the right files.

The folder also determines where new files get created. When Claude Code builds your website, the HTML, CSS, and JavaScript files land inside the project folder. When you want to find those files later, in Finder or in the Terminal, they are all in one place. No hunting around your computer for scattered files.



Starting Claude Code in Your Project

The workflow is always the same two steps:

1. Navigate to your project folder.

```
cd my-website
```

2. Start Claude Code.

```
claude
```

That is it. Claude Code reads the folder, understands what is there (or that nothing is there yet), and waits for your instructions.

If the folder is empty, Claude Code knows you are starting from scratch. If the folder already has files in it from a previous session, Claude Code picks up where you left off. It reads the existing files, understands the structure, and can continue building from where you stopped.

This is one of the things that makes Claude Code different from a regular chatbot. A chatbot forgets everything when you close the window. Claude Code looks at the files in your folder every time you start it. Your project is the memory.

The Two-Command Habit

Every time you sit down to work on something, you will type the same two commands:

```
cd my-website
claude
```

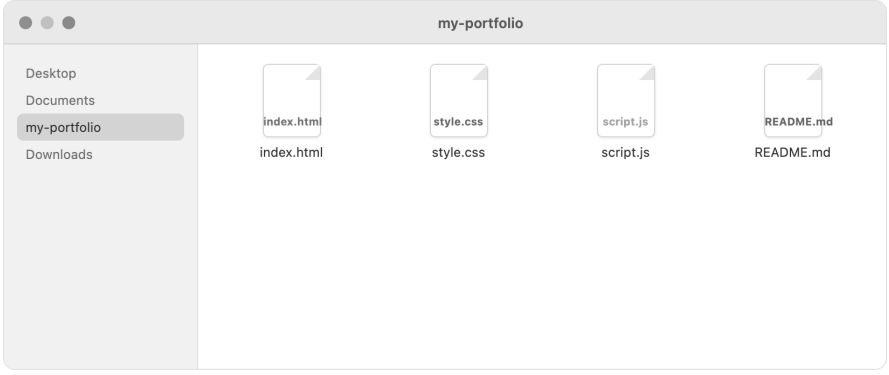
Navigate. Start. It becomes muscle memory fast. You do not need to remember anything else about the Terminal to get started with Claude Code.

> **Tip:**
> If you forget what folders you have, type `ls` to list everything in your current location. You will see your project folders right there.

One Folder, One Project

Keep things simple. One project gets one folder. Do not mix two unrelated things in the same folder. Claude Code works best when it has a clean, focused workspace, just like you do.

Later in the book, we will talk about teaching Claude Code about your project with a special file called CLAUDE.md. That file lives inside your project folder too. Everything stays together, everything stays organized.



Finder window showing a project folder with several files (index.html, style.css, script.js) that Claude Code created, alongside the Terminal window open to the same folder

For now, the thing to remember is: a project is a folder, and Claude Code works inside whatever folder you point it at. The

next section covers one more concept before you start building: what source code actually is, and why you do not need to understand it.

8

What Is Source Code?

When Claude Code builds something for you, it creates files. Those files contain source code. Before you see the word "code" and feel your eyes glaze over, take a breath. You do not need to understand source code to use Claude Code. But knowing what it is, at a very basic level, will make the rest of this book make more sense.

Text Files With Specific Names

Source code is just text. Plain, readable text stored in files. The only thing that makes a source code file different from a regular text file is its extension, the part after the dot in the filename.

You already understand file extensions. A file ending in `.jpg` is a photo. A file ending in `.pdf` is a document. A file ending in `.mp3` is a song. Source code works the same way. The extension tells the computer what kind of code is inside.

Here are the ones you will encounter most often:

`.html` is the structure of a web page. It defines what is on the page: headings, paragraphs, images, links. Think of it as the

blueprint of a building. It says "there is a wall here, a door there, a window over there."

`.css` is how the page looks. Colors, fonts, spacing, layout. If HTML is the blueprint, CSS is the interior designer. Same building, but now the walls are painted, the furniture is placed, the lighting is set.

`.js` stands for JavaScript, and it controls what the page does. Buttons that respond when you click them. Forms that check your input. Menus that slide open. If HTML is the structure and CSS is the look, JavaScript is the behavior.

`.py` stands for Python. It is used for tools, automation, data processing, and backend systems. You will see it if you ask Claude Code to build something that works behind the scenes rather than in a browser.

There are dozens of other extensions, but these four cover most of what a beginner will encounter.

How Files Work Together

A website is rarely a single file. Most websites are a collection of files that work together. The HTML file references the CSS file to know what styling to apply. It might reference a JavaScript file to know what behavior to add. It might reference image files to display photos.

When Claude Code builds a website for you, it creates this collection and connects everything automatically. You do not need to tell it which files to link together. It knows that

`index.html` needs to reference `style.css`, and it writes the connection into the code.

This is worth knowing because when you look at your project folder after Claude Code finishes building, you will see multiple files. That is normal. They are not separate projects. They are different parts of the same thing, like chapters in a book. Each file has a role, and together they create the finished product.

If Claude Code creates a file you do not recognize, you can always ask: "What is the script.js file for?" and it will explain. You do not need to open the file and read the contents. Just ask.

You Do Not Need to Read This Stuff

This is the important part. When Claude Code creates an `.html` file or a `.js` file, you do not need to open it and try to understand what is inside. Claude Code wrote it. Claude Code understands it. If something needs to change, you tell Claude Code what you want, and it makes the edit.

The source code is for the computer, not for you.

Your job is to describe what you want in plain language. Claude Code translates your description into the specific text files that make it happen. You are the person who says "I want a blue front door." Claude Code is the one who knows which file to open and which line to change.

Source Files vs. What You See

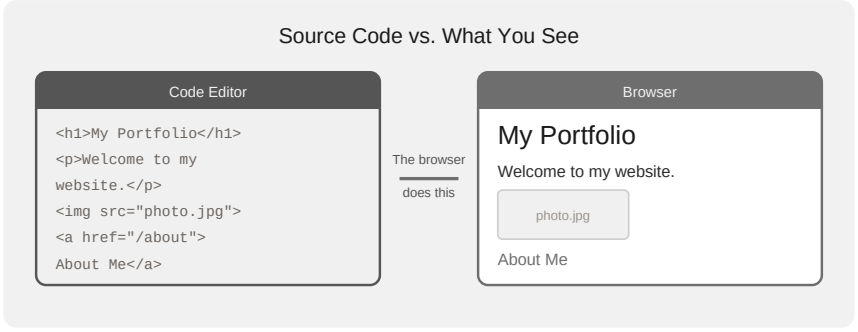
There is one more concept worth understanding: the difference between the source files and the finished product.

When Claude Code creates your website, it generates files like `index.html`, `style.css`, and maybe `script.js`. If you open `index.html` in a text editor, you will see something like this:

```
<h1>Your Name</h1>
<p>Welcome to my portfolio.</p>
```

That looks nothing like a website. It is just tagged text.

But when you open that same file in a web browser, like Chrome or Safari, the browser reads those tags and renders them visually. The `<h1>` becomes a big bold heading. The `<p>` becomes a paragraph. The CSS file tells the browser what colors to use and how to arrange things on screen.



The source files are the recipe. The browser is the kitchen. You see the finished meal.

A Quick Peek at What Code Looks Like

You will never need to write code yourself, but seeing a tiny example makes the whole concept less mysterious. Here is what a few lines of CSS look like:

```
body {
  background-color: #1a1a2e;
  color: #ffffff;
  font-family: Arial, sans-serif;
}
```

Even without knowing CSS, you can probably guess what this does. The background color is set to a dark blue. The text color is white. The font is Arial. Code is not as cryptic as it looks from a distance. Some of it reads almost like English.

You do not need to write this. You do not need to read it when Claude Code shows it to you. But if you happen to glance at a few lines and recognize what they do, that is a natural side effect of working with Claude Code. Some people find they start understanding bits of code over time, without trying. Others never look at it at all. Both approaches work.

Why This Matters

You will see Claude Code creating and editing files during your sessions. It might say something like "I've created `index.html` and `style.css`." Now you know what those are. You will see

file names with extensions you recognize. You will understand that `.html` is structure, `.css` is styling, and `.js` is behavior.

You still do not need to read the contents of those files. But you are no longer in the dark about what they are.

In the next section, you are going to build something real. Claude Code will create source files for you, and you will open the result in your browser. That is the section where all of this theory becomes practice, and where the ten minutes you spend will produce something you can actually show people.

9

Building a Personal Website

This is it. Everything you have read so far, the Terminal, the commands, the concept of projects and source code, it all leads here. You are about to build a real website. Your website. From scratch. Without writing a single line of code yourself.

If you follow along with this section, you will have a working personal portfolio page on your screen in about ten minutes. Not a mockup. Not a screenshot. A real website, running in your browser, that you can show to other people.

What You Are Building

A clean, modern personal portfolio page. It will have your name (we will use a placeholder for now), a short bio section, a list of your skills, and links to your social profiles. Nothing complicated. Something that looks like you hired a web developer to make it, except you did it by having a conversation.

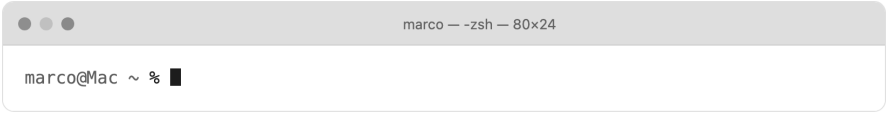
Ready? Open your laptop.

Step 1: Open Terminal

Press `Cmd` + `Space` on your Mac. Type "Terminal." Press Enter.

The Terminal window appears with its blinking cursor. By now, this should feel familiar. If it does not, flip back to Section 2 for a quick refresher.

You are going to type a few commands here before handing things over to Claude Code. These are the same commands you already learned: making a folder and navigating into it.



A freshly opened Terminal window on macOS, showing the default prompt

Step 2: Create a Project Folder

Remember from Section 7: a project is just a folder. You need a folder for your new website. Type the following and press Enter:

```
mkdir my-portfolio && cd my-portfolio
```

Two things just happened. `mkdir my-portfolio` created a new, empty folder called `my-portfolio`. `cd my-portfolio` moved you inside that folder. The `&&` connects the two commands so they run one after the other.

To confirm you are in the right place, type:

```
pwd
```

You should see something like `/Users/yourname/my-portfolio`. That is your project folder. It is empty right now, but not for long.

Terminal showing the mkdir and cd commands, followed by pwd output showing the path to my-portfolio

Step 3: Start Claude Code

This is the last Terminal command you need. Type:

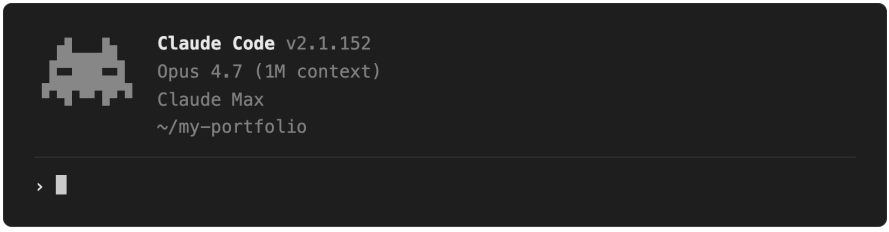
```
claude
```

Press Enter.

Claude Code starts up. You will see a welcome screen with some information about the current session, like which AI model it is using and what folder it is looking at. At the bottom, there is a prompt where you can type messages. The cursor is blinking, waiting for your input.

You are now inside Claude Code, and Claude Code is looking at your `my-portfolio` folder. It knows the folder is empty. It is ready to build whatever you describe.

Take a second to notice how simple the interface is. No menus, no toolbars, no panels on the side. Just a text area where you type and a space where Claude responds. If you have ever used a messaging app, you already know how this works. The only difference is that the person on the other end can build things on your computer.



Claude Code welcome screen in Terminal, showing the session info and the empty prompt at the bottom

Step 4: Describe What You Want

This is the step that matters most. You are going to tell Claude Code what to build, in plain language, the same way you would describe it to a friend.

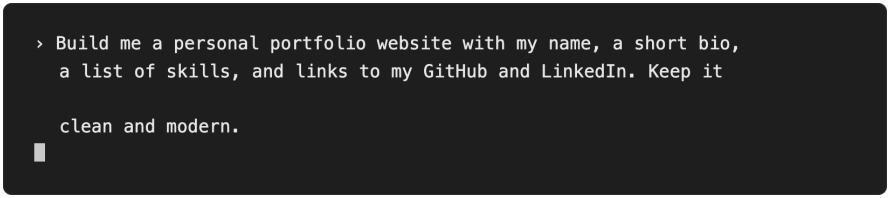
Type this:

```
Build me a personal portfolio website. Include my name (use
placeholder), a short bio section, a list of skills, and lir
profiles. Make it look clean and modern.
```

Press Enter.

Read that prompt again. There is nothing technical in it. No HTML tags. No CSS properties. No JavaScript functions. Just a description of what you want the website to contain and how you want it to feel.

That is your only contribution to the code itself. Everything that happens next is Claude Code doing the building.



The prompt typed into Claude Code, showing the full request text

> **Tip:**
> You do not have to use this exact wording. "Make me a simple website with my name and some info about me" works too. Claude Code is good at interpreting what you mean. Be as specific as you want, or keep it general, either way it will produce something.

Step 5: Watch Claude Code Work

After you press Enter, Claude Code starts thinking. You will see it processing your request, planning what to build, deciding which files to create.

Then it starts creating files. This is the part that feels closest to watching something being built in real time. Claude Code writes out the content of each file, and you can see the text scrolling by.

For a simple portfolio site, it will typically create two or three files:

- `index.html`, the structure of your page. This is the file that your browser will open. It contains the headings, paragraphs, links, and overall layout.
- `style.css`, the visual design. This file controls what the page looks like: colors, fonts, spacing, how elements are arranged on screen.
- `script.js` (sometimes), for interactive elements. If Claude Code adds a smooth scroll effect, a mobile menu, or anything that responds to clicks, it goes here.

You will see the contents of these files appear in your Terminal as Claude Code writes them. Green-highlighted text means code being added. It will look like a wall of unfamiliar symbols and tags. That is normal. You do not need to read it, understand it, or check it for errors.

What matters is that Claude Code is being transparent. It is showing you exactly what it is creating, not hiding anything. Over time, you might start recognizing patterns in the code, like `<h1>` meaning a big heading, or `color:` meaning a color value. That comes naturally with exposure, but it is never required.

Claude Code mid-build, showing the creation of index.html with green highlighted new code. The terminal shows several lines of HTML being generated.

Step 6: Approve the Files

Claude Code does not just silently dump files onto your hard drive. It asks permission before creating or modifying any file. This is a safety feature, so nothing changes on your computer without you agreeing to it.

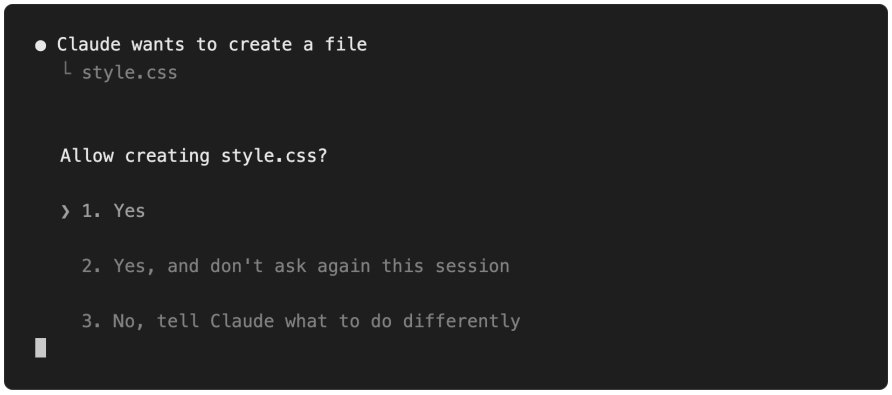
You will see prompts that look like:

```
Allow creating index.html? (y/n)
```

Type `y` and press Enter. Claude Code saves the file and moves on to the next one.

You will get one of these prompts for each file Claude Code creates. For a simple website, that means two or three approvals. Each one takes a second.

> **Tip:**
> If you want to skip the individual approvals and just let Claude Code work, press `Shift + Tab` to switch to "auto-accept edits" mode. Claude Code will still show you what it is creating, but it will not pause for permission on each file. For your first project, approving one by one is a good way to see exactly what is happening. After a few projects, you will probably switch to auto-accept for convenience.



A permission prompt in Claude Code showing "Allow creating style.css?" with the cursor ready for the user to type y or n

After you have approved all the files, Claude Code will give you a summary. Something like: "I've created a portfolio website with your name, bio, skills, and social links. The files are index.html, style.css, and script.js."

Your project folder is no longer empty. It has a website in it.

Step 7: Open the Result

This is the moment. You want to see your website in a browser.

The easiest approach: just ask Claude Code to do it. Type:

```
Open the website in my browser
```

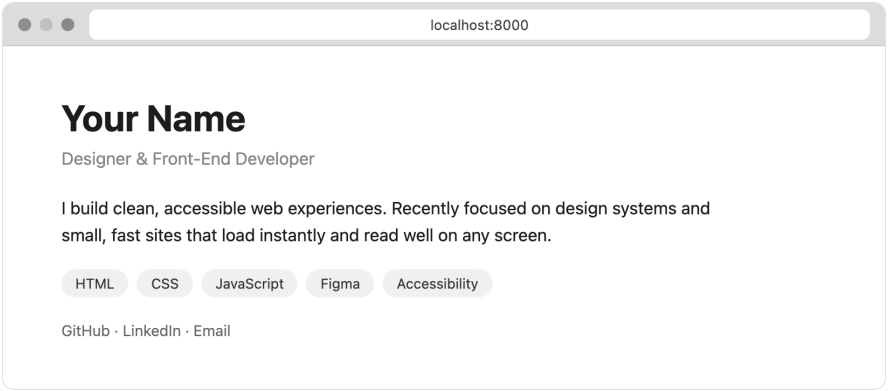
Claude Code will run the command `open index.html` for you (it will ask permission to run this command first, type `y` to approve). Your default browser, Chrome, Safari, Firefox, whatever you use, will open with your portfolio page loaded.

If you prefer doing it yourself, press `Ctrl + D` to exit Claude Code (you can always start it again), and type directly in Terminal:

```
open index.html
```

Your browser opens. And there it is.

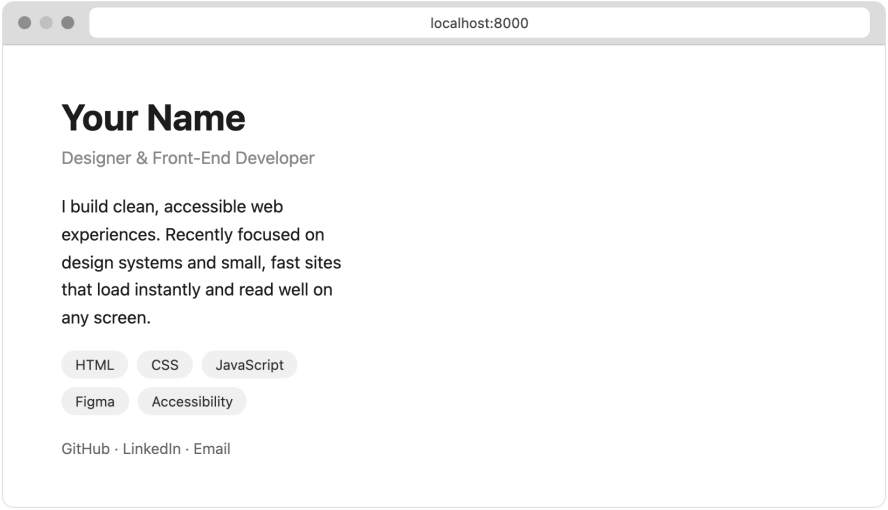
A real website. Your name at the top in a clean, bold heading. A bio section with placeholder text. A list of skills. Social links at the bottom. Clean design, proper spacing, readable fonts. It does not look like a school project or a template. It looks like something that was built by a professional, because in a sense it was. Claude Code knows good web design patterns, the kind of spacing and typography that makes a page look polished.



A clean, modern portfolio page displayed in Chrome or Safari. The page shows "Your Name" as a large heading, a short bio paragraph below it, a section with skills displayed as tags or a clean list, and social media links at the bottom. The design is minimal with good typography.

This is the moment that tends to shift how people feel about Claude Code. Before this, it was abstract. You knew it could do things, but you had not seen the result. Now you have a web page on your screen that did not exist five minutes ago. You described it in one sentence, and it appeared.

Take a moment to scroll through it. Click the links (they will be placeholder URLs, but they work as links). Resize the browser window and watch the layout adjust. This is a responsive website, meaning it adapts to different screen sizes automatically. Make the window narrow, like a phone screen, and watch the layout reflow. That responsiveness happened without you asking for it. Claude Code builds it in by default because that is how modern websites should work.



The same portfolio page viewed in a narrow browser window, showing how the layout adapts to a phone-sized screen

You built this. With one sentence and a few keystrokes.

Step 8: Make It Yours

The first version is a starting point, not the final product. That is completely normal and expected. No one, not even professional developers, gets everything right on the first pass. The real power of Claude Code is how easy it is to iterate.

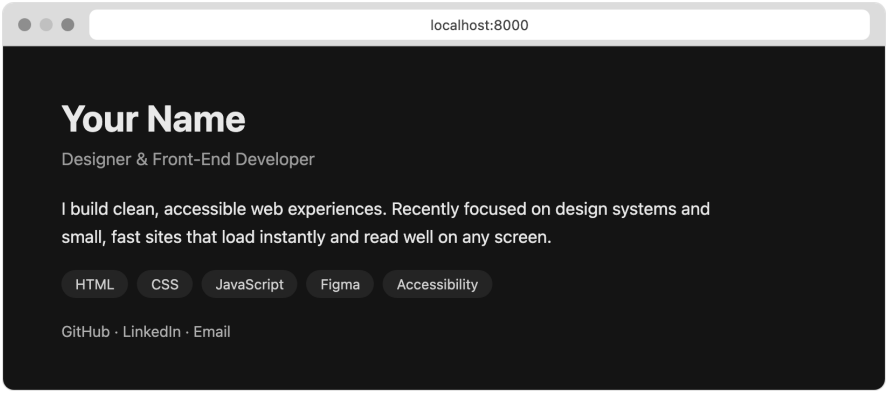
Go back to Claude Code. If you exited, just type `claude` in Terminal (make sure you are still in the `my-portfolio` folder). If you never left, just type your next request.

Here is what a real iteration session looks like. Try these one at a time:

Change the color scheme:

```
Make the background dark with light text
```

Claude Code edits `style.css`, changing background colors and text colors. Approve the change, refresh your browser. The whole mood of the site shifts in seconds.



The portfolio page now with a dark background (dark gray or navy) and light text, showing a dramatic visual difference from the default version

Add more content:

```
Add a "Projects" section with three placeholder project card
should have a title, short description, and a link.
```

Claude Code adds a new section to `index.html` and styles it in `style.css`. You now have project cards on your portfolio.

Change the typography:

```
Use a more playful font for the headings
```

Claude Code might link a Google Font and update the CSS. Your headings get a fresh look.

Add a profile photo:

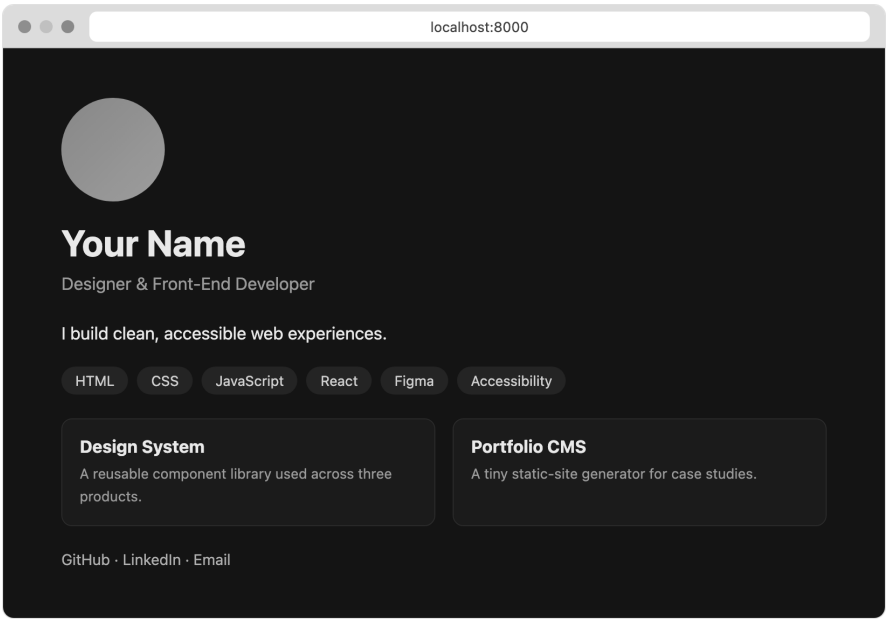
```
Add a circular profile photo at the top of the page. Use a p
image for now.
```

Claude Code adds an image element with a circular crop and uses a placeholder service so you can see exactly where the photo will go and how big it will be.

Rearrange the layout:

```
Make the skills section display as a two-column grid instead
```

The skills section gets restructured. Same content, different presentation.



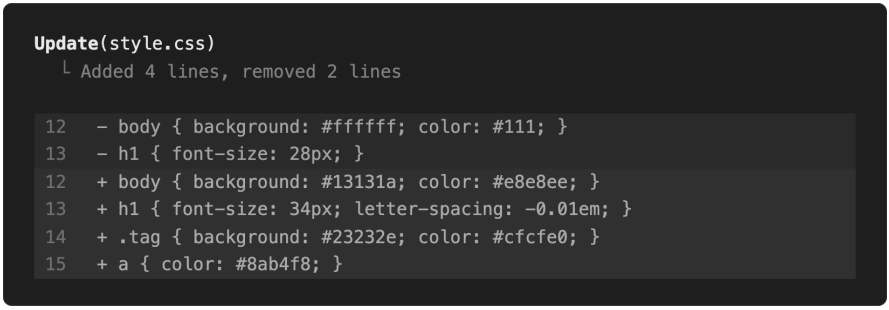
The portfolio page after several iterations, now with dark mode, project cards, a circular placeholder photo, and skills displayed in a grid. The page looks substantially different from the first version.

Each of these changes takes seconds to request and seconds for Claude Code to implement. You describe what you want, Claude Code makes the edit, you approve, you refresh the browser. The cycle is fast and natural.

Notice that you do not need to know which file to edit, or where in the file the change should happen. You say "make the skills a two-column grid" and Claude Code figures out which lines of CSS to change. You stay in the world of describing and reviewing. The technical details stay on Claude Code's side of the conversation.

If you make a request and do not like the result, just say so. "Undo that, I preferred the list" works. "That font is hard to read, go back to the previous one" works. Nothing is permanent

until you decide you are happy with it.



Claude Code showing a diff view for one of the changes. Green lines show new code being added. Red lines show old code being removed. A clear visual representation of what changed.

The Full Conversation

Here is what the entire conversation flow looks like from start to finish, condensed into one view:

```
You: Build me a personal portfolio website. Include my name
     Name" as placeholder), a short bio section, a list of s
     links to social profiles. Make it look clean and modern

Claude: I'll create a modern portfolio website for you. Let
        files...
        [Creates index.html, style.css, script.js]

You: Open the website in my browser

Claude: [Runs: open index.html]

You: Make the background dark with light text

Claude: I'll update the color scheme...
        [Edits style.css]

You: Add a Projects section with three placeholder cards

Claude: [Edits index.html and style.css]

You: The project cards are too close together, add more spac

Claude: [Edits style.css, increases margin between cards]

You: Looks good. Add a contact form at the bottom with name,
     and message fields.

Claude: [Edits index.html and style.css]
```

Notice the rhythm. You describe. Claude Code builds or edits. You review. You describe the next thing. It feels like working with someone, not operating a tool.

Saving Your Work

Your project files are saved in the `my-portfolio` folder on your computer. They are not going anywhere. Close the Terminal, shut down your Mac, come back next week, the files are still there.

The next time you want to work on your website, use the two-command habit from Section 7: `cd my-portfolio`, then `claude`. Claude Code opens inside the folder and sees all the files from your last session. You pick up right where you left off. "Add a blog section" or "Change the font size" or "Make the contact form actually send emails." The conversation continues.

> **Tip:**
> If you want to save snapshots of your project as it evolves, ask Claude Code: "Initialize a git repository for this project." Git is a version control system that tracks every change you make over time. You do not need to understand how it works, Claude Code handles it for you. It means you can always go back to a previous version if you do not like a change.

What Just Happened

Take a second to register what you just did.

You opened Terminal. Created a folder. Typed one sentence describing what you wanted. Got a working website. Then you refined it through a normal conversation, making changes in

plain English, watching the results appear in your browser.

You did not learn HTML. You did not learn CSS. You did not learn JavaScript. You did not install a code editor, follow a tutorial, or read documentation.

You described what you wanted, and it appeared.

The next section covers what "running your project" means in more detail, including what happens when `open index.html` is not enough and you need something called a development server. It is simpler than it sounds, and Claude Code handles most of it for you.

10

Running Your Project

You have built a website. You opened it in your browser. It worked. But what does "running" code actually mean? And why is it sometimes as simple as double-clicking a file, but other times involves starting something called a "server"?

This section clears that up.

What "Running Code" Means

When you "run" code, you are telling your computer to read a set of instructions and execute them. That is all. A source code file sitting on your hard drive is just text. It does nothing on its own. Something needs to read it and act on it.

For a website, that "something" is your web browser. Chrome, Safari, Firefox, they all know how to read HTML, CSS, and JavaScript files and turn them into the visual pages you interact with every day.

For other types of code, like a Python script, the "something" is a program called an interpreter. It reads the Python file and does whatever the file says to do.

The good news: you almost never need to think about this. Claude Code handles the details. But understanding the basics will help when things do not work the way you expect.

The Simple Way: Just Open the File

For the portfolio website you built in the previous section, running it was straightforward:

```
open index.html
```

This tells your Mac to open the file in your default browser. The browser reads the HTML, applies the CSS styling, runs any JavaScript, and displays the result. Done.

You can also find the file in Finder (it is in your `my-portfolio` folder) and double-click it. Same thing happens.

This approach works for any project that is made up of plain HTML, CSS, and JavaScript files. No additional steps required. These are called "static" files because they do not change, your browser just reads them and displays them as-is.

Most simple projects, and the ones you will build early on, work this way.

When You Need a Server

At some point, you will build something that does not work by just double-clicking the HTML file. You will open it in your browser and get a blank page, or things will look broken, or some features will not work.

This is not an error on your part. Some projects are designed to be served, not opened directly. Think of it like the difference between reading a book and watching a movie. A book works on its own. A movie needs a player. Some code works on its own in a browser. Other code needs a server to "play" it.

This usually happens for one of two reasons.

The project uses a framework. If you ask Claude Code to build something with React, Vue, Next.js, or similar tools, the source files need to be processed before a browser can understand them. The raw code is not ready to be displayed directly. A development server does this processing in real time, translating the source code into something the browser can render.

The project needs backend logic. If your project talks to a database, sends emails, processes forms, or does anything that requires server-side work, it needs a running process to handle those tasks. A browser alone cannot do this. The browser is the front door, but the server is the building behind it.

You do not need to memorize these reasons. The key thing to know is: if double-clicking the file does not work, you probably need a development server. And you do not need to figure out which server or how to set it up. Claude Code handles that.

Asking Claude Code to Start a Server

When you need a server, ask Claude Code. It is the simplest approach.

```
Start a development server so I can see this in my browser
```

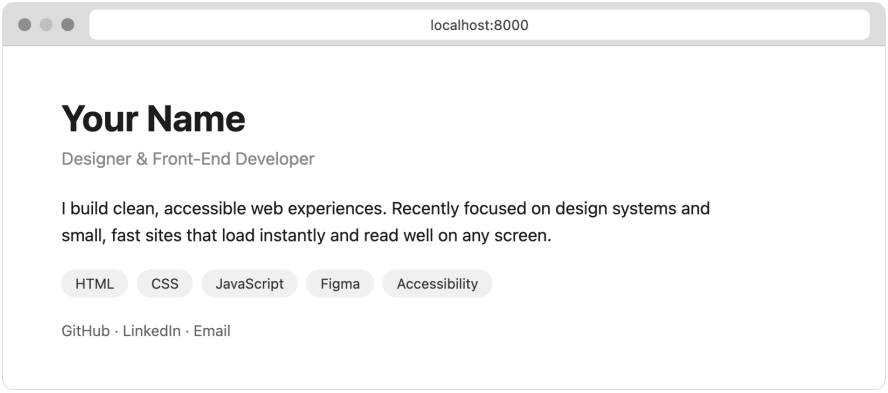
Claude Code will figure out what kind of server your project needs and start it. It might install a lightweight server, or it might use one that came with the framework it used to build your project. Either way, it handles the setup.

After starting the server, Claude Code will tell you something like:

```
Server running at http://localhost:3000
```

Open your browser, type `localhost:3000` into the address bar, and you will see your project.

Terminal showing Claude Code's output after starting a development server, with the localhost URL visible



A browser address bar showing "localhost:3000" with a web page loaded below it

What Is localhost?

That `localhost:3000` address looks unfamiliar if you have never seen it before. It is simpler than it seems.

`localhost` means "this computer." Not a website out on the internet. Just your Mac, right here. When you type `localhost` in your browser, you are telling it: "Do not go to the internet. Load something running right here on this machine."

The `:3000` is a port number. Think of ports like doors on a building. Your computer has thousands of ports, and a server picks one to listen on. Port 3000 is a common default. Sometimes you will see 8080 or 5173 or other numbers. They all work the same way.

So `localhost:3000` means: "Go to this computer, door number 3000, and show me what is there."

> **Note:**
> The address `http://127.0.0.1:3000` means exactly the same thing as `http://localhost:3000`. If you see either one, they point to the same place.

Stopping a Server

A running server keeps running until you stop it. If you close the Terminal window, the server stops. If you want to stop it without closing Terminal, press `Ctrl + C` in the Terminal window where the server is running.

This is a common source of confusion for beginners: you start a server, close the Terminal, then wonder why the website stopped working. The server was running inside that Terminal process. When the window closed, the server shut down with it. This is normal and expected. Just start it again next time you want to work on the project.

One thing to keep in mind: while the server is running, that Terminal window is "busy." It is dedicated to running the server and you cannot type other commands in it. If you need to use Terminal for something else, open a second Terminal window (`Cmd + N`). Your server keeps running in the first window while you work in the second.

The Workflow for Server-Based Projects

For projects that need a server, the workflow has one extra step compared to the simple HTML approach:

1. Navigate to your project folder.

```
cd my-portfolio
```

2. Start Claude Code.

```
claude
```

3. Ask Claude Code to start the server.

```
Start the dev server
```

4. Open your browser to the address Claude gives you (usually localhost:3000 or similar).

5. Make changes through Claude Code. Many development servers automatically refresh the browser when files change. You ask Claude Code to make an edit, the server detects the change, and your browser updates on its own.

When Things Look Different From Expected

Sometimes you open your project and something looks off. The page loads, but the layout is wrong, or an image is missing, or a feature does not work. Before assuming you did something wrong, try these quick checks:

Hard refresh the browser. Press `Cmd + Shift + R`. Browsers sometimes cache old versions of files and show you stale content. A hard refresh forces the browser to load everything fresh.

Check the right address. If you are using a development server, make sure your browser is pointing to `localhost:3000` (or whatever address Claude Code gave you), not at a file path. Opening the file directly and loading it through a server can produce different results.

Ask Claude Code. Describe what you see. "The page is blank" or "The menu is not showing up" or "The background color did not change." Claude Code can diagnose and fix the issue.

You Do Not Need to Remember the Commands

If you take one thing away from this section, make it this: you do not need to memorize how to start servers, which port to use, or what framework requires what tool. Ask Claude Code.

"How do I see this in my browser?"

"Start a development server for this project."

"The page is blank when I open it, what do I need to do?"

Claude Code knows. It will set things up and tell you where to look. Your job is to describe what you want and review the results. The plumbing is handled for you.

With your project running in the browser, you have the full loop: create, view, change, view again. That cycle is the foundation for everything you build from here on out. The next section covers how to get better at the most important part of that loop: describing what you want.

PART 4

WORKING WITH CLAUDE CODE

11

How to Ask for Things

The single most important skill when using Claude Code is knowing how to describe what you want. Not how to code it. Not the technical details of how it should be implemented. Just what the end result should look and feel like.

This is a shift from how most people think about working with computers. You are used to needing the right button, the right menu, the right setting. With Claude Code, you skip all of that. You just talk.

Describe the What, Not the How

The biggest mistake beginners make is trying to sound technical. They think Claude Code needs instructions in

computer language. It does not. In fact, the more you try to dictate the technical approach, the worse your results will be, because you are guessing at implementation details you do not know.

Here is a bad prompt:

```
Create a div with flexbox, center-aligned items, three child
with border-radius 8px and box-shadow, responsive grid at 76
```

Unless you already know CSS, this is nonsense to you. And if you do not know CSS, you are probably guessing at what these words mean. You are writing instructions for a tool without knowing how the tool works. That is a recipe for frustration.

Here is a good prompt for the same thing:

```
I need a section on my website that shows three features sid
Each feature should have an icon at the top, a short title,
description underneath. On phones, they should stack vertica
of sitting next to each other.
```

This prompt is better in every way. It describes the result, not the method. Claude Code knows what CSS properties to use. It knows about responsive design. It knows about flexbox and grid. Let it make those decisions. Your job is to be clear about what you want the person visiting your website to see.

Being Specific Matters

There is a big gap between a vague request and a specific one, and that gap directly affects how useful Claude Code's first attempt will be.

Compare these two prompts:

Vague: "Add a contact form to my website."

Specific: "Add a contact form to my website with fields for name, email, and message. The email field should check that it is a real email address. When someone submits the form, show a green success message that says 'Thanks, I will get back to you within 24 hours.' The form should match the style of the rest of the site."

The vague version will get you a contact form. It might have two fields or five. It might look completely different from your existing design. Claude Code did not do anything wrong, you just did not give it enough to work with.

The specific version tells Claude Code exactly what fields to include, what validation to add, what feedback to show, and that it should match your existing style. The first attempt will be much closer to what you had in mind.

You do not need to get every detail right upfront. But the more specific you are, the fewer rounds of back-and-forth you will need.

Show What You Mean

Sometimes the fastest way to communicate what you want is to point to something that already exists.

```
I want the navigation bar to feel similar to the one on stri
clean and minimal with just a logo on the left and a few lir
the right. Nothing fancy, no dropdown menus.
```

You do not need to link to the website (Claude Code cannot browse the internet during a session). Just describe what you remember about it. The goal is to give Claude Code a mental picture of the direction you want.

This also works for more abstract things:

```
I want this page to feel calm and professional. Think law fi
startup. Muted colors, lots of white space, traditional-look
```

Claude Code understands tone and style. It can translate "calm and professional" into specific design choices. You do not need to know the hex code for slate blue.

Iteration Is Normal

Your first result will probably not be perfect. That is completely expected, and it is not a failure. Even professional designers work in iterations, building something, reviewing it, adjusting, and repeating.

The way you iterate with Claude Code is just by talking to it:

```
The layout looks good but the text is too small. Make the bo
bigger and add more spacing between the sections.
```

```
I like the colors but the header feels too tall. Can you mak
more compact?
```

```
The contact form works but the button is hard to see. Make i
out more, maybe use the same blue as the header.
```

Each of these is a small, clear adjustment. You are looking at the result, deciding what to change, and describing the change in plain language. That is the entire workflow.

Some projects take one prompt. Some take twenty rounds of refinement. Both are normal. The conversation history means Claude Code remembers everything you have discussed, so it understands the context. You do not have to repeat yourself.

Real Examples, Before and After

Here are a few more examples of how to rephrase vague prompts into specific ones.

Before: "Make a homepage." **After:** "Build a homepage for my freelance photography business. I want a large hero image at the top, a short paragraph about what I do, a grid of my best photos (I will add the images later, just use placeholders for now), and a way for people to contact me at the bottom."

Before: "Add a dark mode." **After:** "Add a toggle in the top right corner that switches between light and dark mode. In dark mode, the background should be dark gray (not pure black) and the text should be off-white. Remember which mode the visitor chose so it stays the same when they come back."

Before: "Make it look better." **After:** "The content feels cramped. Add more spacing between sections, increase the line height of the body text, and make the headings a bit larger. Keep the same fonts and colors."

Notice the pattern. The good versions describe what the result should look like or how it should behave. They do not mention HTML tags, CSS properties, or JavaScript functions. You are giving Claude Code a clear target, and it figures out how to hit it.

When You Are Not Sure What You Want

Sometimes you do not have a clear picture of the end result. You know you want a website, but you are not sure what it should look like. You know you need a tool, but you are not sure what features it should have.

That is fine. Start broad and narrow down. You can say:

```
I need a landing page for my tutoring business. I am not sur
what it should look like. Can you build a simple version and
tell you what to change?
```

Claude Code will make reasonable default choices and produce something you can react to. It is much easier to look at a concrete result and say "I want this different" than to describe the perfect version from scratch. Use the first draft as a starting point, not a final product.

You can also ask Claude Code for suggestions: "What sections should a tutoring business website have?" It will propose a structure based on what works for similar sites. You pick the parts you like and discard the rest.

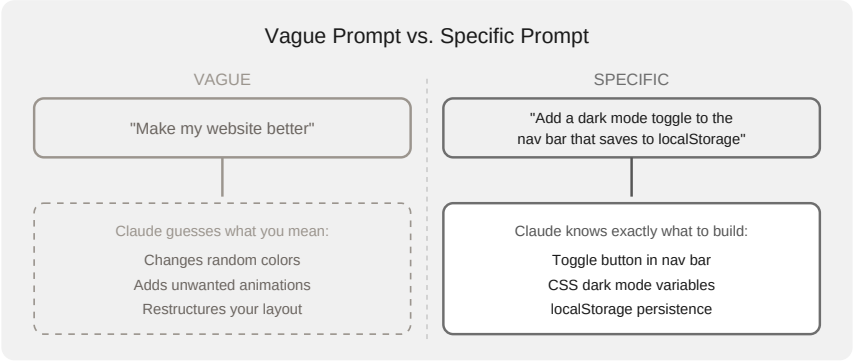
The Conversation Is the Tool

Here is something that might take a minute to sink in: with Claude Code, your ability to describe what you want in everyday language is your programming skill. That is not a metaphor. The conversation is literally how you build things.

The people who get the most out of Claude Code are not the ones with the most technical vocabulary. They are the ones who are clear about what they want, specific about the details that matter to them, and willing to look at the result and say what needs to change.

You already know how to do all three of those things. You do them every time you explain to a contractor what you want in a home renovation, or tell a graphic designer what kind of logo you are looking for, or describe a dish you want to a waiter at a restaurant.

The only difference is that your conversation partner is an AI agent that can turn your description into working code in seconds. The skill is the same. The speed is just different.



You now know how to describe what you want. The next section covers how to read what Claude Code shows you back, the visual signals that tell you what is changing, what is being created, and when you need to make a decision.

12

Understanding What Claude Shows You

When Claude Code works on your project, it does not do everything silently in the background. It shows you what it is doing, what it wants to change, and asks for your approval before making modifications. This section explains what you are looking at so none of it feels mysterious.

You do not need to understand the code itself. You just need to understand the interface, the few visual signals that tell you what is happening.

The Diff View: Green and Red

When Claude Code edits a file, it shows you a "diff," which is a before-and-after view of what changed. The concept is simple:

Green lines are things that were added. New code, new content, new functionality.

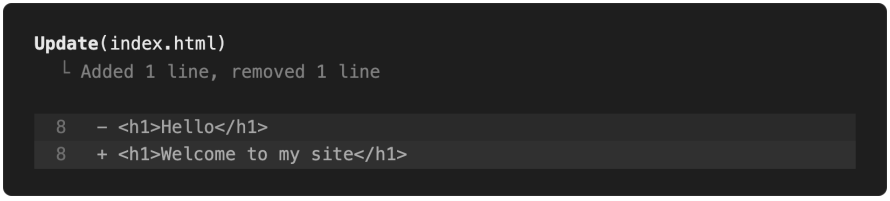
Red lines are things that were removed. Old code that got replaced or deleted.

If you see mostly green, Claude Code is adding new stuff. If you see a mix of green and red, it is replacing something that was there before with something new. If you see mostly red, it is removing something you no longer need.

You do not need to read the actual code on those lines. The color alone tells you the scope of the change. A small edit will show one or two lines changing. A big restructuring will show large blocks of red and green.

This gives you a useful gut check. If you asked Claude Code to "change the heading color to blue" and the diff shows two lines changing, that looks right. If the same request shows fifty lines changing, something might be off. You can ask Claude Code what it is doing before approving, or simply deny the change and rephrase your request.

Over time, you will develop an intuition for how big a diff should look relative to what you asked for. A one-sentence request usually means a small diff. A "redesign the whole page" request means a large one. When the size of the diff does not match the size of your request, it is worth pausing to ask questions.



A diff showing a few lines of green (added) and red (removed) in a file edit

When Claude Code Creates Files

Sometimes instead of editing an existing file, Claude Code creates a new one. When this happens, it tells you the file name and shows you the contents. Since everything is new, the entire file appears in green.

This is common when you start a project from scratch. Your first conversation might result in Claude Code creating several files: an HTML file for the structure, a CSS file for the styling, maybe a JavaScript file for interactivity. Each one gets its own creation notification.

Again, you do not need to read the code. Just note what files were created. If you asked for a personal website and Claude Code created `index.html`, `style.css`, and `script.js`, that is exactly what you would expect. The names are descriptive enough.

When Claude Code Runs Commands

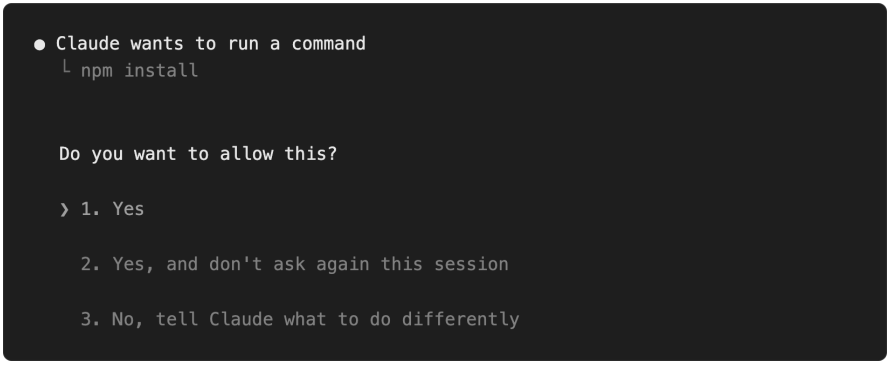
Claude Code sometimes needs to run commands on your behalf. Installing a package, starting a server, creating folders, or running a test. When it does this, you will see the command it wants to run, displayed in a distinct format so you can tell it apart from normal text.

The prompt will show the exact command and ask for your permission. Something like:

```
Run command: npm install express
```

You do not need to understand what the command does. If you asked Claude Code to "set up a server" and it wants to run `npm install express`, that makes sense in context. If it wants to run something you did not ask for and cannot explain, deny it and ask what it is trying to do.

After a command runs, you will see the output in the terminal. This might be a success message, a progress indicator, or occasionally an error. Claude Code reads this output too and responds accordingly. If a command fails, Claude Code will typically explain what went wrong and try a different approach.



Claude Code showing a command it wants to run, with the permission prompt visible

Claude Code's Text Responses

Between the diffs and the commands, Claude Code also responds with plain text. It explains what it is about to do, summarizes what it just did, and answers your questions. These

text responses appear in a slightly different style from the code changes, usually without the green/red highlighting.

Pay attention to these responses. They often contain useful information: "I created three files for your website," "I noticed your CSS had a conflict and fixed it," "The server is now running at localhost:3000." This is Claude Code keeping you informed, the same way a contractor might say "I moved the light switch because the original spot would have been behind the door."

If Claude Code says something you do not understand, ask it to clarify. "What do you mean by that?" or "Why did you do it that way?" are both valid follow-ups. Claude Code will explain in plain language.

The Thinking Indicator

When you send a message to Claude Code, there is usually a brief pause before it responds. During this time, you will see a small thinking indicator, a spinner or animated dots that show Claude Code is working on your request.

Short requests (like changing a color) usually take a few seconds. Bigger requests (like building an entire website from scratch) might take thirty seconds or more. This is normal. Claude Code is reading your project files, figuring out what to do, and writing the code, all before it shows you the result.

Do not type another message while Claude Code is thinking. Wait for it to finish. If it seems stuck for more than a minute or two, press `Ctrl + C` to cancel and try rephrasing your request.

```
✳ Thinking… (esc to interrupt · 8s)
```

Claude Code's thinking indicator showing that it is processing a request

Permission Prompts

Claude Code asks before it does things. This is a deliberate safety feature, not an annoyance. Before it writes to a file, runs a command, or makes changes to your project, you will see a prompt asking for your approval.

The options are straightforward:

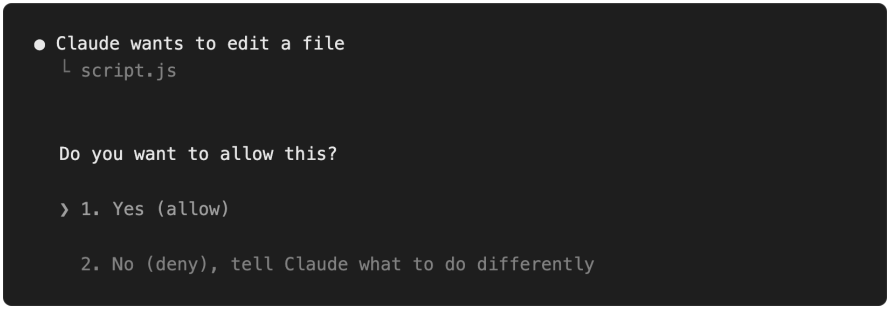
Allow means yes, go ahead and do it. Claude Code will proceed with the action it described.

Deny means no, do not do it. Claude Code will skip that action and ask you what you would like to do instead. Nothing bad happens when you deny something. You are just telling Claude Code to try a different approach.

When you see a permission prompt, it will describe what Claude Code wants to do. Something like "Edit index.html" or "Run npm start." Read the description, and if it matches what you expect, allow it. If it wants to do something you did not ask for, deny it and ask why.

You will also sometimes see an option to allow a specific type of action for the rest of the session. This saves you from approving the same kind of edit twenty times in a row. If Claude Code is making a series of changes to your CSS file and you trust the

direction, you can allow that once and let it continue without interruption.



A permission prompt showing the action Claude Code wants to take, with Allow and Deny options

A Quick Refresher on Modes

You learned about the three permission modes in Section 6: Normal, Auto-accept, and Plan mode, cycled with **Shift+Tab**. Now that you have been building for a few sections, here is when each mode becomes most practical:

Normal mode is still best when you are trying something new or making changes you want to review carefully. You see each diff before it happens.

Auto-accept mode is where most people end up during active building sessions. Once you trust the direction and the diffs are matching your expectations, switching to auto-accept lets Claude Code work without pausing for each file edit. You still see what is happening, you just do not have to approve each step individually.

Plan mode becomes useful when your project has grown and you want Claude Code to analyze the current state before making changes. "What would need to change if I wanted to add a dark mode?" is a good plan-mode question. Claude Code reads, thinks, and proposes, but does not touch anything until you switch back.

> **Tip:**
> You can always check which mode you are in by looking at the bottom of the Claude Code interface. The current mode is displayed there.

You now understand what Claude Code is showing you and how to control its level of autonomy. The next section covers what to do when things do not go as planned, because they will, and that is completely normal.

13

When Things Go Wrong

Things will go wrong. Not because you did something stupid, but because that is how working with computers has always been. Professional developers hit errors every single day. Errors are not a sign that you are failing. They are a normal, expected part of the process.

The good news is that Claude Code is remarkably good at fixing its own mistakes. Most of the time, your job is just to tell it what happened.

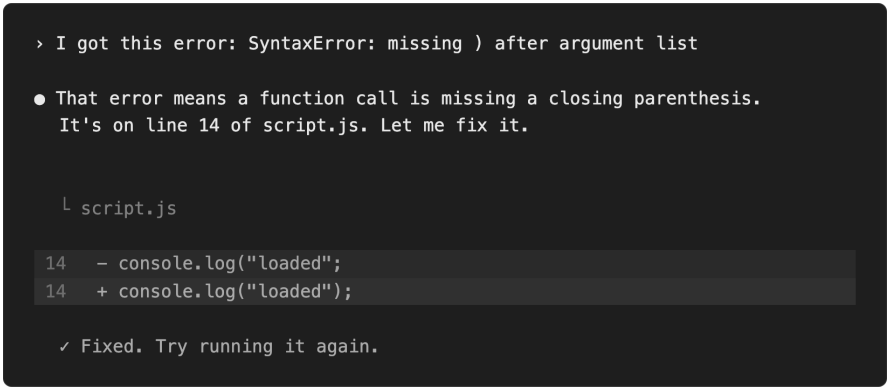
Error Messages Are Not Personal

When you see a red error message in the terminal, your instinct might be to panic or assume you broke something. Resist that instinct. An error message is just a computer telling you what went wrong, in its own clumsy language. And Claude Code speaks that language fluently.

The single most useful thing you can do when you hit an error: copy the error message and paste it back to Claude Code. That is it. Say something like:

```
I got this error: [paste the error message here]
```

Claude Code has seen thousands of these. It will recognize the error, explain what it means in plain language, and fix it. You do not need to understand the error yourself.



An error message in the terminal, followed by the user pasting it back to Claude Code, which then explains the issue and provides a fix

Common Errors and What They Mean

Here are the errors you are most likely to see as a beginner. None of them are serious.

"Command not found"

This means your computer does not have a specific tool or program installed. Maybe Claude Code tried to use something that is not on your machine yet.

What to do: paste the error back to Claude Code. It will either install the missing tool for you or suggest an alternative approach. This is one of the most common errors and one of the easiest to fix.

"Permission denied"

Your computer has a security system that prevents programs from doing certain things without your explicit approval. This error means Claude Code (or a command it ran) tried to do something the system blocked.

What to do: paste the error back to Claude Code. It will explain what permission is needed and walk you through granting it. This often comes down to a simple approval step. The system is doing its job, protecting you.

"Port already in use"

When you run a website locally on your computer, it uses a "port," which is like a numbered channel. If something else is already using that channel, you get this error. Maybe you have another project running, or a previous session did not shut down cleanly.

What to do: tell Claude Code "the port is already in use." It will either stop whatever is using that port or switch to a different one. If you want to handle it yourself, pressing Ctrl+C in the terminal where the other project is running will usually stop it.

The Screen Looks Garbled or Frozen

Sometimes the terminal output gets messy. Text overlaps, colors look wrong, or nothing seems to respond when you type. This is not a crash, the display just got confused.

What to do: press **Ctrl+C** to stop whatever is running. If that does not help, type `/clear` to reset the display. If even that fails, close the terminal window entirely and open a new one. None of your work is lost. Your files are still on your computer.

The Fresh Start

Sometimes the cleanest solution is to start a new Claude Code session. This is not giving up. It is a legitimate strategy that even experienced developers use regularly.

1. Type `/exit` to close the current session. This ends the conversation.

2. Type `claude` to start fresh. A new session begins with a clean slate.

Your project files are untouched. Nothing gets deleted. You just start a new conversation. If the problem was caused by a confusing back-and-forth that led Claude Code down a wrong path, starting over with a clear, specific prompt often solves it immediately.

If you want to pick up where you left off instead of starting completely fresh, use `claude -c` to continue the most recent conversation (as covered in Section 6).

Telling Claude Code to Fix Its Own Mistakes

Claude Code is not perfect. Sometimes it will make a change that breaks something, or produce a result that does not match what you asked for. That is fine. Just tell it.

```
That broke the layout. The header is now overlapping the cor
Can you fix it?
```

```
The form looks right but the submit button does not do anyth
when I click it.
```

```
This is not what I meant. I wanted the photos in a grid, not
stacked vertically. Like a photo gallery.
```

Be direct. Describe what you see, what you expected, and what is wrong. Claude Code will go back, review what it did, and make corrections. It does not get offended, and it does not need you to be polite about it (though it does not hurt).

The pattern is always the same: describe the problem, let Claude Code handle the fix. You do not need to diagnose the issue yourself.

Common Beginner Mistakes

A few things trip up almost everyone in the first few sessions. Knowing about them in advance will save you time.

Forgetting to navigate to your project folder. Claude Code works in whatever folder you are currently in. If you type `claude` from your home directory instead of your project folder, Claude Code will not see your project files. Always `cd` into your project folder first, or tell Claude Code where your project is.

Trying to edit files that do not exist yet. If you ask Claude Code to modify your website's header but you have not created the website yet, it has nothing to work with. Start with "build me a website" before asking it to change parts of one.

Assuming Claude Code remembers previous sessions. Each new session starts fresh. Claude Code reads your project files, so it understands the current state of your code, but it does not remember the conversation you had yesterday. If you want to continue an old conversation, use `claude -c` to resume it.

Closing the terminal while a server is running. If Claude Code started a local server for you (so you can see your website in a browser) and you close the terminal, the server stops and your website will no longer load in the browser. Just start Claude Code again and ask it to run the server.

When the Result Is Not What You Expected

This is different from an error. Sometimes Claude Code builds exactly what you asked for, but what you asked for is not actually what you wanted. The website works, but the layout feels wrong. The colors are fine, but the overall vibe is off. The form works, but it does not look like what you had in mind.

This is not a bug. It is a communication gap, and it is completely normal. The fix is the same as with any collaborator: give more specific feedback.

Instead of "this does not look right," try "the sections feel too cramped, add more space between them, and make the font size a bit larger." Instead of "I do not like this," try "I was expecting a more minimalist look, fewer colors, more whitespace, smaller headings." The more specific you are about what you want changed, the fewer rounds of iteration you need.

Over time, you will get better at describing what you want on the first try. But even experienced users iterate. That is not a failure of the tool. That is how design works.

The Golden Rule

If you are stuck, just describe what happened. Do not try to fix technical problems yourself. Do not Google error messages. Do not try to edit code files manually. Just tell Claude Code what you see, what you expected, and what went wrong.

```
Something is not working. When I open the page in my browser
a blank white screen instead of my website. Here is what the
says: [paste any messages you see]
```

Claude Code has seen more error messages than you will encounter in a lifetime. It is almost always faster to describe the problem in plain language and let it sort out the solution than to try to debug it yourself.

Errors are not roadblocks. They are speed bumps. And Claude Code is good at driving over speed bumps.

You have the core skills now: building, running, asking, reviewing, and troubleshooting. The next section shows you how to make Claude Code work even better by teaching it about your specific project and preferences.

PART 5

LEVELING UP

14

Teaching Claude About Your Project

Every time you start Claude Code, it walks into your project folder fresh. It can see the files that are there, read them, and figure out what is going on. But it does not know your preferences. It does not know that you hate bright colors, or that you want all your text in a specific font, or that you tried a particular approach last week and it did not work. It starts from zero context every session.

There is a simple fix for this. You leave it a note.

CLAUDE.md: A Note for Your AI Colleague

A CLAUDE.md file is a plain text file that you put in your project folder. When Claude Code starts up, it reads this file first, before it does anything else. Whatever you write in there becomes part of its instructions for the session.

Think of it like leaving a note on the kitchen counter for a house sitter. "The Wi-Fi password is on the fridge. The cat eats at 7. Do not open the back door, it sticks." You are giving context to someone who is competent but unfamiliar with the specifics of your situation.

The file is not code. It is plain English. You do not need any special format or syntax. You just write what you want Claude Code to know.

Where It Lives

Create a file called `CLAUDE.md` in the top level of your project folder. If your project is in `my-website`, the file goes at `my-website/CLAUDE.md`.

You can create it from inside Claude Code itself. Just type:

```
/init
```

Claude Code will look at your project, figure out what kind of project it is, and generate a starting CLAUDE.md for you. It reads

the files you already have, notices the patterns, and writes a description that captures the current state of the project. You can then edit it to add your own preferences.

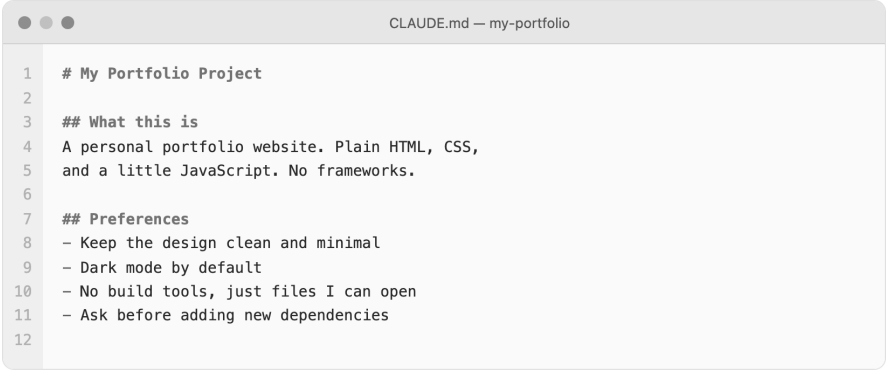
Or you can ask Claude Code to create one from scratch: "Create a CLAUDE.md file that describes this project and my preferences." Tell it what you care about, and it will write the file. Either approach works. The `/init` command is faster if you want a starting point. Asking Claude Code directly gives you more control over what goes in.

What to Put in It

For a beginner project, keep it simple. Here is a real example for a personal portfolio website:

```
# My Portfolio Website

## About This Project
This is my personal portfolio website. It has a homepage wit
a page listing my work, and a contact form.

## My Preferences
- I like dark backgrounds with light text
- Use sans-serif fonts, nothing too fancy
- Keep the design minimal and clean
- The color accent should be teal (#2dd4bf)

## Things to Avoid
- Do not use jQuery, use vanilla JavaScript
- No animations or flashy effects
- Do not add a blog section, I do not want one

## How to Test
Open index.html in a browser to see the site.

## Important Notes
- My name is spelled "Sara" not "Sarah"
- My email is on the contact page, do not change it
```

That is a complete CLAUDE.md file. No code, no special formatting. Just plain statements about what the project is, what you like, what you do not want, and how to check that things work.



A CLAUDE.md file open in a text editor, showing the project description and preferences sections

Notice the sections. You do not need to follow this exact structure, but organizing your notes under headings makes them easier for both you and Claude Code to read. A few clear sections are better than a wall of unorganized text.

What It Changes

Without a CLAUDE.md file, conversations with Claude Code often include a lot of repeated clarification. "Make the background dark." "No, I said sans-serif." "I told you last time I do not want jQuery." Each session, you start that conversation over.

With a CLAUDE.md file, Claude Code already knows your preferences when it starts. It will use dark backgrounds without being asked. It will pick sans-serif fonts by default. It will avoid jQuery. The conversation shifts from correcting and repeating to actually building.

This changes the kind of work you do in each session. Instead of spending the first five minutes reminding Claude Code how your project works, you start immediately with new requests. "Add a contact form to the homepage." "Change the layout to two columns." You jump straight into forward progress.

The effect compounds over time. Each session is more productive because you spend less time re-explaining yourself.

Building It Up Over Time

Your CLAUDE.md file is a living document. You do not need to write the perfect version on day one. Start with the basics, a project description and a few preferences, and add to it as you go.

Every time Claude Code does something you do not like, that is something to add to the file. If it keeps choosing a font you hate, add "Never use Times New Roman." If it keeps adding features you did not ask for, add "Only build what I explicitly ask for. Do not add extra features." If it keeps putting navigation at the bottom of the page when you want it at the top, write that down.

These small additions stack up. After a few sessions, your CLAUDE.md captures the decisions you have already made, so you do not revisit them. After a few weeks, it becomes a detailed manual for how you want things done. The more specific it gets, the less you need to repeat yourself.

Some people update their CLAUDE.md at the end of each session. Others add to it in the moment, when they notice Claude Code doing something wrong. Both work fine. The point is that the file grows with your project.

You Write Context, Not Code

This is a key idea. With CLAUDE.md, your role is not writing code. Your role is writing context. You are describing the project, setting boundaries, and communicating preferences. Claude Code translates all of that into the actual technical decisions.

This is the same skill you use when working with any collaborator. You tell them what you want, what you do not want, and what matters to you. They handle the execution. The CLAUDE.md file just makes that communication persistent, so it survives between sessions.

If you have ever written instructions for a colleague covering your desk while you are away, you already know how to write a CLAUDE.md. It is the same kind of thinking: what does this person need to know to do a good job without me standing over their shoulder?

Auto Memory

Claude Code also learns on its own. As you work together, it automatically saves notes about your project, things like build commands it discovered, patterns it noticed, mistakes it made

and corrected. These notes live in a separate memory system and load alongside your CLAUDE.md file.

You do not need to manage auto memory. It happens in the background. But it means Claude Code gets better at working with your project even beyond what you explicitly write down. If it tried something that failed and then found a better approach, it remembers that for next time.

Think of CLAUDE.md as the instructions you write, and auto memory as the notes Claude Code writes to itself. Both work together to make each session smoother than the last.

A Personal and a Project File

You can also create a personal CLAUDE.md file that applies to all your projects. This one lives at `~/.claude/CLAUDE.md` (in your home directory, inside a hidden folder called `.claude`). Anything you put in that file applies everywhere you use Claude Code.

This is useful for global preferences. Maybe you always want sans-serif fonts, or you never want Claude Code to use a particular library, or you prefer minimal designs in general. Put those in the personal file, and keep project-specific instructions in the project file.

Claude Code reads both. The project file takes priority when the two conflict. If your personal file says "use blue accents" but your project file says "use teal accents," Claude Code will use teal for that project.

> **Tip:**
> You do not need to set up a personal CLAUDE.md right away. Start with a project-level one. Once you notice yourself writing the same preferences across multiple projects, that is when a personal file saves you time.

The Key Takeaway

CLAUDE.md is the simplest way to make Claude Code work better for you. It is not technical. It is not code. It is a note, written in your own words, that tells Claude Code how to be a better collaborator on your specific project.

Start small. Add as you go. The five minutes you spend writing preferences now will save you hours of repeating yourself later.

The next section introduces one more way to save yourself repetition: skills. If CLAUDE.md is your project preferences, skills are your saved workflows, reusable instructions for tasks you do often.

15

Skills: Saving What Works

At some point while using Claude Code, you will find a workflow that works really well. Maybe you figured out the right way to ask for a PDF report, or you dialed in the exact instructions for building a certain kind of web page. You will want to save that so you do not have to explain it again every time.

That is what skills are.

What Is a Skill?

A skill is a saved set of instructions that Claude Code can follow. Think of it like a recipe. Instead of explaining from scratch every time how you want your reports formatted or your web pages structured, you save those instructions once, and Claude Code uses them whenever they are relevant.

A skill is not a program or a piece of code you need to understand. It is a text file with a description and a set of instructions, written in plain English. When you invoke a skill, Claude Code reads those instructions and follows them.

Here is what makes this different from the CLAUDE.md file we discussed in the last section. CLAUDE.md gives Claude Code general context about your project, your preferences and your conventions. A skill gives Claude Code a specific playbook for a specific task. CLAUDE.md says "I prefer minimal designs." A skill says "When I ask for a report, use this exact layout with these fonts and this header format."

Installing Skills from skills.sh

There is an open directory of skills at skills.sh, with thousands of skills created by other people. These cover common tasks that many people need: generating PDFs, creating web designs, writing in specific formats, working with images, and much more.

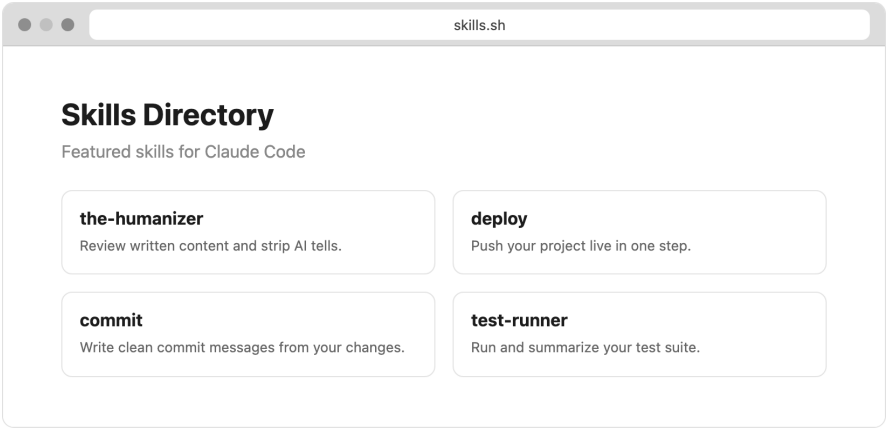
To install a skill from the directory, you run this command in your Terminal:

```
npx skills add owner/skill-name
```

The `owner/skill-name` part is listed on the skills.sh website next to each skill. You browse the directory, find something that matches what you need, and install it with that one command.

Once installed, the skill is available to Claude Code in all your projects. You use it by typing a slash command in Claude Code. If you installed a skill called `pdf-report`, you would type `/pdf-report` and Claude Code knows what to do. It reads the skill's instructions and follows them, just like it reads your

CLAUDE.md file.



The skills.sh directory showing featured skills

> **Note:**
> Skills are created by the community, so quality varies. Stick with popular, well-reviewed skills when starting out. The most-installed skills tend to be the most reliable.

Popular Skills People Use

Some categories that are especially useful for non-developers:

Document creation. Skills that tell Claude Code how to generate PDFs, slide decks, or formatted reports with specific layouts and styling. If you need a professional-looking document and do not want to fuss with formatting, there is probably a skill for it. For example, a PDF skill might specify the exact fonts, margins, header layouts, and color schemes so

every report you generate looks consistent and polished.

Web design. Skills that set design guidelines, like following a particular CSS framework, or building mobile-friendly pages by default. These are useful if you want consistent, good-looking results without specifying every design detail yourself. A Tailwind CSS skill, for instance, tells Claude Code to use that specific styling library and follow its conventions whenever it builds web pages.

Writing assistance. Skills that enforce a specific tone, format, or style across everything Claude Code helps you write. Useful if you produce content regularly and want it to feel consistent. You might install a "blog post" skill that formats every article with a specific heading style, paragraph spacing, and word count range.

Image generation. Skills that connect Claude Code to image creation tools, letting you generate visuals as part of your projects. You describe what you want, and the skill handles the technical details of creating it.

Data and automation. Skills for common data tasks, things like parsing spreadsheets, generating charts, or pulling information from web sources into organized formats. A data-cleaning skill could standardize messy CSV files into a consistent format every time you run it.

Browsing and Installing

The skills.sh website organizes skills by category and popularity. Each skill listing shows a description of what it does,

the install command, and reviews from other users. Browsing it is a good way to discover workflows you had not thought of. Even if you do not install anything right away, seeing what other people have built gives you ideas for your own projects.

You do not need to understand how these skills work internally. You install them, invoke them with a slash command, and Claude Code handles the rest.

Creating Your Own Skill

The easiest way to create a skill is to just ask Claude Code to do it. After you have a workflow that works well, say something like:

"Turn the way we just built that report into a skill I can reuse."

Claude Code will create a skill folder with a SKILL.md file containing the instructions. That file captures the workflow so you can repeat it later with a single command. You do not write the skill yourself. You describe what you want saved, and Claude Code turns it into the right format.

Skills live in your `.claude/skills/` folder, either in your home directory (for skills that work across all projects) or inside a specific project folder (for project-specific skills). The structure looks like this:

```
.claude/skills/my-skill/
  SKILL.md
```

The SKILL.md file contains a name, a description of when to use the skill, and the instructions Claude Code should follow. It is all plain text. You can open it, read it, and edit it if you want to adjust the behavior later. If the skill almost does what you want but not quite, you can tweak the instructions yourself or ask Claude Code to update them.

Here is a practical example. Say you build a weekly status report for your team. Every Friday, you ask Claude Code to create a PDF with a summary of what happened that week. After doing this three times, you realize you give the same formatting instructions each time: specific fonts, a company logo at the top, sections for accomplishments and next steps.

That is a perfect candidate for a skill. Ask Claude Code to save those instructions, and next Friday you just type `/weekly-report` and the formatting is already handled. You only need to provide the content.

Skills Work Across Tools

One thing worth knowing: skills follow an open standard called Agent Skills. This means a skill you create for Claude Code also works with other AI coding tools that support the same standard, including tools like Cursor, Codex, and others. If you ever switch tools or use multiple ones, your skills come with you.

This is not something you need to worry about now, but it is good to know that the effort you put into creating skills is not locked into a single tool.

You Do Not Need Skills to Start

Skills are a convenience, not a requirement. You can use Claude Code for months without ever installing or creating a skill. They become useful once you notice yourself repeating the same instructions across multiple sessions or projects.

The signal is repetition. If you find yourself typing the same kind of instructions for the third or fourth time, that is the moment to save it as a skill. Future you will appreciate not having to explain the same thing again.

You now have the complete toolkit: Claude Code itself, CLAUDE.md for project context, and skills for reusable workflows. The final section looks ahead at what you can build with all of this.

16

What You Can Build From Here

If you have followed along from the beginning, you now know more than most people about using an AI coding agent. You know how to open a terminal, install Claude Code, start a project, describe what you want, review changes, troubleshoot errors, and save your workflows. That is a genuine set of skills, and it is worth taking a moment to recognize that.

The question is: what do you do with them?

Project Ideas

Here are real things that non-developers are building with Claude Code. All of these are within reach with what you have learned so far.

A personal blog. A static website with your writing, organized by date or topic. No hosting costs if you use a free platform like GitHub Pages or Netlify. You write the content, Claude Code builds the site. When you want a new post, you ask Claude Code to add one. Over time, you end up with a full archive of your

writing that you own and control.

A small business landing page. A professional-looking single page for your freelance work, consulting practice, or local business. Contact form included. Hours of operation. A map showing your location. Photos of your work or your space. The kind of thing that would cost you several hundred dollars from a web designer, built in an afternoon.

A calculator or conversion tool for your specific work. Maybe you are a contractor who estimates project costs based on square footage and materials. Maybe you run a tutoring business and need to calculate hourly rates across different packages. Maybe you manage events and need a tool that splits costs across attendees with different meal choices. These niche tools are perfect for Claude Code because they are too specific for anyone to have built already, but simple enough to describe in a conversation.

A recipe collection website. Your family recipes, organized by category, searchable, with photos. A project that is both practical and personal. You can share the link with family members, and adding new recipes is as simple as telling Claude Code "add my grandmother's soup recipe" and giving it the ingredients and steps.

A simple game. A trivia quiz on a topic you love. A word puzzle. A memory matching game. A flashcard system for something you are studying. These are fun to build and fun to share. Nothing that will compete with a console game, but something you made yourself that actually works in a browser.

An automation script for something you do weekly. Renaming batches of files. Reformatting data from one spreadsheet layout to another. Pulling information from one place and organizing it somewhere else. Sorting photos into folders by date. If you do something repetitive on your computer, there is probably a script that can do it for you in seconds.

None of these projects require you to learn programming. They require you to describe what you want clearly and work with Claude Code to iterate until it is right. That process, the back-and-forth of describing, reviewing, and refining, is the core skill you have been building throughout this book.

What Keeps Things Realistic

A note on expectations. Claude Code is powerful, but there are boundaries worth knowing about.

It works best for self-contained projects. A personal website, a standalone tool, a local script. These are ideal. Building something that needs to integrate with complex existing systems, like plugging into a company's internal database or connecting to an enterprise software platform, involves challenges that go beyond what this book covers.

It works best when you can see and test the result yourself. If you ask for a website, you can open it in a browser and check immediately. If you ask for a data script, you can run it and see the output. That feedback loop, where you look at the result and tell Claude Code what to change, is what makes the process work.

It works less well for things you cannot evaluate. If you have no way to tell whether the result is correct, you have no way to guide Claude Code toward a better result. Stick with projects where you can look at the output and say "yes, that is what I wanted" or "no, change this part."

One more thing. The projects you build will be real and functional, but they will also be simple by professional standards. A personal website built with Claude Code is a real website that works in a browser. It is not the same as a website built by a team of developers over six months. That is fine. Most of the time, the simple version is exactly what you need.

Putting Your Project on the Internet

Everything you have built so far lives on your computer. Other people cannot see it unless they are sitting at your desk. The next natural step is putting your project online, and Claude Code can help with that too.

The simplest approach is to ask: "How do I publish this website so other people can see it?" Claude Code will walk you through the process. For a basic HTML website, it usually involves pushing your files to a free hosting platform like GitHub Pages, Netlify, or Vercel. These platforms take your project files and make them available at a public web address that anyone can visit.

You do not need to understand the details of hosting. You just need to follow the steps Claude Code gives you, which typically involve creating a free account on a hosting platform and running a few commands. The whole process takes about ten minutes, and once it is set up, updates are just as easy.

The result: a real URL, something like `yourname.netlify.app`, that you can share with anyone. Your portfolio, your business page, your tool, live on the internet, accessible from any device.

Where to Go Next

This book focused on Claude Code in the terminal, which is the most capable way to use it. But there are other options as your comfort grows.

Cowork. Claude has a non-terminal agent designed for working with files and browsing the web. If you prefer not to use the terminal at all, Cowork provides a more visual interface for many of the same tasks. It runs alongside Claude's regular web interface, and you can use it to work on documents, research topics, and build things without opening Terminal.

The Claude Code documentation. The official docs at code.claude.com/docs cover everything in more detail, including features we did not touch in this book. Topics like model switching, background tasks, advanced permissions, and team collaboration are all documented there. As you get more comfortable, the docs become a useful reference for expanding what you can do.

Community. Other people are using Claude Code for all kinds of things, and many of them are not developers either. Online communities, forums, and social media groups are good places to see what others are building, ask questions, and share your own work. Seeing other people's projects is one of the best ways to get ideas for your own.

The Skill You Actually Learned

This book taught you how to use Claude Code, but the underlying skill is broader than that. You learned how to take an idea and translate it into instructions that an AI agent can execute. You learned how to evaluate results, give feedback, and iterate toward something that works. You learned that the terminal is not a locked door, it is just a different kind of window.

Those skills transfer. AI tools will keep evolving. The interfaces will change. New capabilities will arrive. But the ability to clearly describe what you want, evaluate what you get, and work through problems methodically, that stays useful regardless of which specific tool you are using next year or five years from now.

You also learned something about yourself. You can work with tools that feel unfamiliar. You can get through error messages. You can build things you did not think you were capable of building. That confidence is not tied to any particular software.

One Last Thing

You just learned something that most people have not tried. Not because it is hard, but because they assumed it was. They saw a terminal window and assumed it was not for them. They heard "coding agent" and assumed you needed to be a coder.

You now know that is not true.

The terminal is a text box. Claude Code is a collaborator that speaks your language. The code it writes is a means to an end, and the end is whatever you decide to build.

That is the whole book. Go build something.